Year: 2026-2027 Sem V-NEP
Course: Agile Software Development & DevOps Lab
Date of Submission: 11/10/2026

A Mini Project Report on

LabLend

A Lab Equipment Issue and Return Tracker Delivered Through an Agile and DevOps Workflow

Submitted in partial fulfilment of the requirements of the course
Agile Software Development & DevOps Lab
Semester V (NEP)
Academic Year 2026–2027

Submitted by

Name	Roll No.
Kushagra Mehta	2401064
Siddharth Maru	2401060
Varun Masand	2401062

Under the guidance of
Prof. ____Sanober Sheikh_____
Department of ____AI & DS_____
_____Thadomal_____ College of Engineering
2026–2027

CERTIFICATE

This is to certify that the mini project entitled “LabLend: A Lab Equipment Issue and Return Tracker Delivered Through an Agile and DevOps Workflow” is a bona fide work of Kushagra Mehta (2401064), Siddharth Maru (2401060) and Varun Masand (2401062), submitted in partial fulfilment of the requirements of the course Agile Software Development & DevOps Lab, Semester V, during the academic year 2026–2027.

_____ _____

Project Guide Head of Department

Date: _____

Place: _____

ABSTRACT

LabLend is a Lab Equipment Issue and Return Tracker developed to demonstrate how the concepts learned in the ASD & DevOps laboratory can be integrated into a single working software project. It replaces the paper register used to lend laboratory equipment such as Arduino kits, sensors, multimeters and Raspberry Pi boards to students. The application was developed using Jinja2 templates, HTML and CSS for the frontend, Python Flask with SQLAlchemy for the backend, and PostgreSQL as the database in the containerized environment. Students can register and log in with their roll number, browse and search equipment with live availability, request items for 1 to 14 days and view their loans and due dates, while the lab assistant can approve or reject requests, issue items, mark returns, manage equipment stock, view a dashboard of overdue loans and export the loan register as CSV.

The project progressively applies the seven laboratory experiments. Git and GitHub provide source-code and version management, including feature branches, a pull request and a rollback using git revert; Docker packages the Flask application into a portable multi-stage image; Docker Compose runs the application, PostgreSQL, Prometheus, node-exporter and Grafana as interconnected services; Jenkins provides Continuous Integration and is extended to automate Docker image building, deployment and a smoke test; Jira demonstrates an Agile Scrum workflow with epics, stories, sprints and dependencies; and Prometheus with Grafana provides application and server monitoring through the /metrics endpoint. This produces a unified software-development and DevOps workflow rather than isolated demonstrations of individual tools.

Keywords: Agile, Git, GitHub, Docker, Docker Compose, Jenkins, CI/CD, Jira, Prometheus, Grafana, Flask, PostgreSQL

CONTENTS

		Page No.
Chapter 1	Introduction	5
Chapter 2	Requirements and Agile Planning (Experiment 6)	7
Chapter 3	System Design	9
Chapter 4	Application Implementation	11
Chapter 5	DevOps Implementation (Experiments 1–7)	13
Chapter 6	Testing	37
Chapter 7	Implementation	39
Chapter 8	Results and Discussion	43
Chapter 9	Conclusion	44

1. Introduction

1.1 Background

LabLend is a web-based Lab Equipment Issue and Return Tracker developed as the mini project for the ASD & DevOps laboratory. In most college laboratories, equipment issued to students is recorded in a paper register, which makes it hard to know what is available, who holds which item and which items are overdue. LabLend moves this register online and provides a practical context in which the concepts taught through the laboratory experiments can be applied progressively to one application.

The laboratory covers version control, containerization, multi-service deployment, continuous integration, automated deployment, Agile project management and continuous monitoring. LabLend brings these practices together so that development, deployment, planning and monitoring form one connected lifecycle.

1.2 Problem Statement

To design and develop a web-based Lab Equipment Issue and Return Tracker and deliver it through an integrated Agile and DevOps workflow in which source code is version-controlled, the application is containerized, dependent services are orchestrated, changes are automatically tested and deployed, development tasks are tracked through Jira, and the running application and server are monitored.

1.3 Objectives

1. Develop a lab equipment lending application with student and lab-assistant functionality.
2. Allow students to register, log in with their roll number, browse equipment with live availability, request items for a number of days and view their loans and due dates.
3. Allow the lab assistant to approve or reject requests, issue items, record returns, manage equipment stock, see overdue loans on a dashboard and export the loan register.
4. Use Git and GitHub for source-code version control, branching, commits, pull requests, merging and rollback.
5. Containerize the Flask application using Docker and run the application, PostgreSQL and the monitoring stack as interconnected services using Docker Compose.
6. Configure Jenkins for Continuous Integration and extend the pipeline for automated Docker image building, deployment and smoke testing.
7. Use Jira to represent an Agile Scrum workflow with epics, stories, tasks, sprints and dependencies.
8. Integrate Prometheus and Grafana to collect and visualise application and server (CPU, memory, disk) metrics.

1.4 Scope

The project scope is the LabLend web application and the DevOps workflow demonstrated through Experiments 1 to 7. The report focuses on the student and lab-assistant functions and on integrating Git, GitHub, Docker, Docker Compose, Jenkins, Jira, Prometheus and Grafana.

1.5 Mapping of Lab Outcomes

The lab outcomes of the Agile Software Development & DevOps Lab are achieved through the following parts of the LabLend project:

LO	Lab Outcome	Exp.	Achieved in LabLend through	Sections
LO1	Apply Agile methodologies and use Jira for sprint planning, task tracking, and integrating with DevOps workflows.	6	Jira Scrum project SCRUM with 5 epics and 19 stories/tasks, three sprints, story points, dependencies and the To Do → In Progress → In Review → Done workflow linked to Git and Jenkins	2.4, 2.5, 5.6
LO2	Use Git effectively for version control, source code management, and collaborative team development.	1	Feature branches merged with --no-ff, Pull Requests #1 and #2, commit history on GitHub and rollback with git revert	5.1
LO3	Build and manage automated CI/CD pipelines for application development, testing, packaging, and deployment.	4, 5	Jenkins pipeline from the Jenkinsfile: checkout, test image, 22 pytest tests with JUnit report, Docker image build, Compose deployment and /health smoke test; builds #1–#4 including a failed build and its rollback	5.4, 5.5, 6.1, 6.2
LO4	Containerize, orchestrate, and deploy applications using Docker, Docker Compose, Kubernetes, and cloud infrastructure tools.	2, 3	Multi-stage Dockerfile, container lifecycle, and a five-service Docker Compose stack with volumes, health check and service networking	5.2, 5.3
LO5	Implement ML lifecycle management and automated workflows using MLflow, Airflow, and monitoring tools like Prometheus/Grafana/Nagios.	7	Prometheus scraping /metrics and node-exporter, PromQL queries and a Grafana dashboard for application status and CPU, memory and disk	5.7, 6.3

LO	Lab Outcome	Exp.	Achieved in LabLend through	Sections
LO6	Develop and demonstrate an end-to-end DevOps pipeline mini-project integrating all stages of DevOps from code commit to deployment and monitoring.	Mini project	LabLend as one connected pipeline: a commit pushed to GitHub is tested and packaged by Jenkins, deployed with Docker Compose, checked by a smoke test and monitored in Prometheus and Grafana, and demonstrated on the running application	3.1, 5.5, 7, 8

Table 1.1: Mapping of lab outcomes to the LabLend project

2. Requirements and Agile Planning

2.1 User Roles

Role	Main Functionality
Student	Register/login with roll number, browse and search equipment with live availability, request items for 1–14 days and view own loans and due dates
Lab Assistant	Dashboard of available, issued, pending and overdue items; approve or reject requests; mark returns; add equipment and change stock; view and export full loan history

Table 2.1: User roles

2.2 Functional Requirements

- Students can register and log in to the system using their roll number.
- Students can browse and search available equipment with live availability.
- Students can request equipment for a quantity and a number of days (maximum 14 days).
- A student can have at most 3 active requests or loans at a time.
- Students can view their loans, statuses and due dates.
- The lab assistant can approve or reject requests; stock is re-checked at approval.
- The lab assistant can mark items as returned, which restores available stock.
- Issued items past their due date are automatically flagged as OVERDUE.
- The lab assistant can add equipment and change stock (not below the units currently issued).
- The lab assistant can view the full loan history and download it as a CSV file.

2.3 Non-Functional / DevOps Requirements

- The application should run in a consistent environment through Docker.
- Flask and PostgreSQL should communicate as interconnected services through Docker Compose, with persistent database storage.
- Jenkins should automate source retrieval, building and testing, with deployment and a smoke test added to the extended pipeline.
- Jira should provide an Agile/Scrum workflow for development tasks.
- Prometheus should collect application metrics exposed by the Flask application and server metrics from node-exporter, and Grafana should visualise them.

2.4 Scrum-Style Workflow

LabLend used Jira as the Agile planning and tracking tool. The backlog was prepared as five epics (Equipment Catalogue, Issue & Return Workflow, Overdue Tracking & Dashboard, DevOps Pipeline and Monitoring) with 19 stories and tasks, estimated in story points and divided into three sprints in the Jira project `SCRUM` (the initial plan is kept in `jira/LabLend_Jira_Import.csv` in the repository). A Scrum-style workflow was followed in which each issue moves through the following stages:

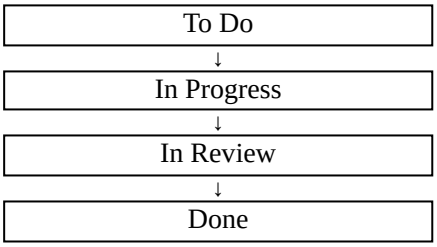


Figure 2.1: Scrum-style workflow followed for LabLend issues

Features were developed using the Git/GitHub workflow, tested, pushed to the repository, reviewed through a pull request and then verified through Jenkins; the matching Jira issue is moved to Done once that verification succeeds.

2.5 Agile-to-DevOps Traceability

Stage	Tool / Practice	Outcome
Planning	Jira	Epics, stories and tasks defined, estimated and planned into sprints
Development	Git / GitHub	Source code versioned on feature branches and merged through pull requests
Verification	Jenkins	Automated build and 22 unit tests executed on every change to main
Deployment	Docker / Docker Compose	Application deployed in containers and checked through /health

Stage	Tool / Practice	Outcome
Monitoring	Prometheus / Grafana	Application and server metrics collected and visualised
Completion	Jira	Task moved to Done after successful verification

Table 2.2: Agile and DevOps workflow traceability

3. System Design

3.1 Architecture

LabLend consists of a server-rendered frontend, a Python Flask backend and a PostgreSQL database. The application and database are deployed in containers. Jenkins manages the automated delivery workflow, Jira manages Agile planning, and Prometheus with Grafana monitors the running application and the host server. The overall integrated workflow is:

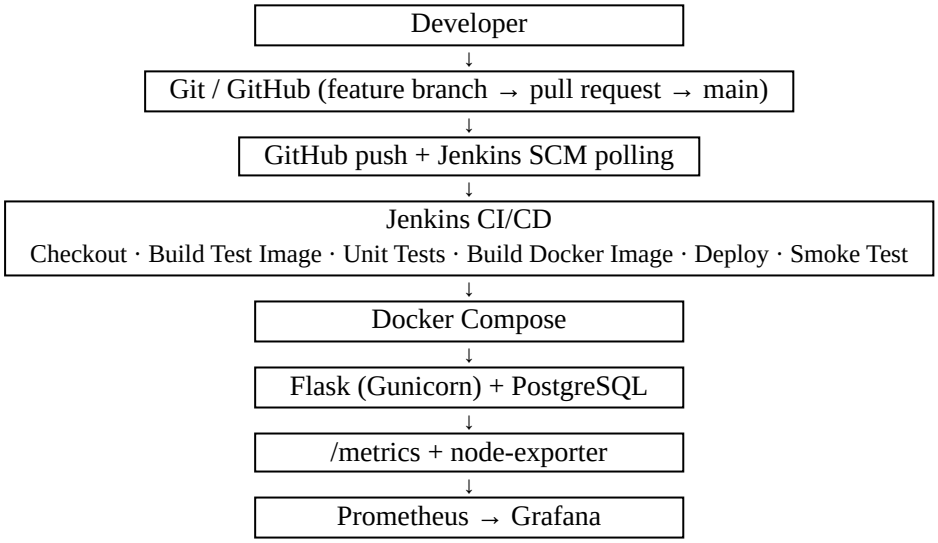


Figure 3.1: Integrated LabLend Agile and DevOps workflow

Jira → Agile planning and task tracking

3.2 Technology Stack

Layer	Technology	Purpose
Frontend	Jinja2 templates, HTML, CSS	Provides the web interface for students and the lab assistant
Backend	Python 3.12, Flask, SQLAlchemy, Gunicorn	Implements the application logic, business rules and JSON API
Database	PostgreSQL 16 (SQLite for local tests)	Stores users, equipment and loans

Layer	Technology	Purpose
Testing	pytest	Automated unit tests for authentication, loan workflow, overdue logic, CSV export and metrics
Version Control	Git and GitHub	Tracks changes, branches, commits, pull requests and remote source code
Containerization	Docker (multi-stage)	Packages the Flask application and its dependencies
Orchestration	Docker Compose	Runs web, db, prometheus, node-exporter and grafana as interconnected services
CI/CD	Jenkins (pipeline as code, configuration as code)	Automates testing, Docker build, deployment and smoke test
Agile Management	Jira	Plans and tracks development tasks
Monitoring	Prometheus, node-exporter, Grafana	Collects application metrics from /metrics and server CPU, memory and disk metrics

Table 3.1: Technology stack

3.3 Service Design

Service	Responsibility
web (Flask + Gunicorn)	Runs the LabLend application on port 5000 and exposes the application pages, /api, /health and /metrics
db (PostgreSQL 16)	Stores LabLend data in the named volume pgdata; a pg_isready health check gates the web service
prometheus	Scrapes metrics from web:5000, node-exporter:9100 and itself every 10 seconds and supports PromQL analysis
node-exporter	Exposes host CPU, memory and disk metrics
grafana	Shows the provisioned dashboard “LabLend – Application & Server Monitoring”

Table 3.2: Service design

3.4 Monitoring Design

The prometheus-flask-exporter package was integrated with Flask to expose HTTP metrics (`flask_http_request_total` and a request-duration histogram) through the /metrics endpoint. Custom business metrics were added in `app/metrics.py`: `lablend_equipment_units{state}`, `lablend_loans{status=requested|issued|overdue}`,

`lablend_loans_issued_total` and `lablend_loans_returned_total`. Prometheus and node-exporter were added to the Docker Compose environment and access the Flask service through the Compose network.

The Prometheus target page was used to verify that the LabLend application, node-exporter and Prometheus were UP. PromQL queries were used to analyse HTTP requests by status and the number of loans by status, while the Grafana dashboard combines application status, overdue loans, pending requests, request rate, p95 latency and server CPU, memory and disk usage.

4. Application Implementation

4.1 Application Features

The LabLend application provides separate functionality for students and the lab assistant. Students can register and log in with their roll number, browse and search equipment with live availability, request items for 1 to 14 days and view their loans and due dates. The lab assistant can see a dashboard of available, issued, pending and overdue items, approve or reject requests, mark returns, add equipment, change stock, view the full loan history and download it as CSV. Business rules: at most 3 active requests/loans per student, loan period of 1–14 days, stock re-checked at approval, quantity cannot go below units issued, and items past the due date are shown as OVERDUE.

4.2 Student Workflow

The student-side workflow begins with registration/login, followed by browsing the equipment catalogue and sending a request. The student can then follow the request and loan status and the due date.

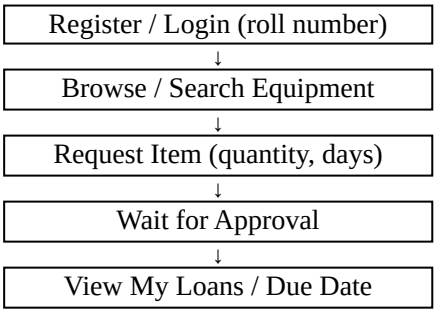


Figure 4.1: Student workflow

4.3 Lab Assistant Workflow

The lab assistant reviews pending requests, issues the equipment, records returns and keeps the equipment list up to date. This provides a separate administrative workflow from the student request process.

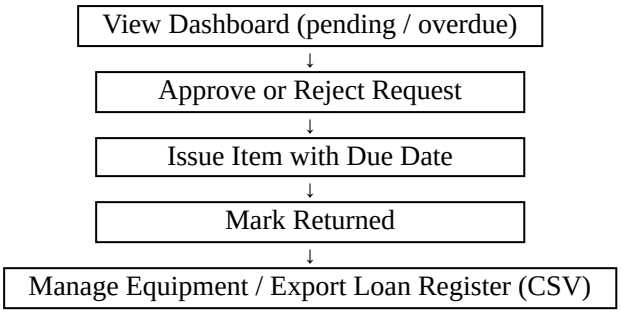


Figure 4.2: Lab assistant workflow

4.4 Project Repository and Application Structure

The Git repository maintains the Flask backend, HTML templates, static files, Docker configuration, Jenkinsfile, tests, Jira backlog and monitoring configuration. These project files were committed and pushed to the GitHub repository `kushagra819/ASD-D` as part of Experiment 1.

```
ASD-D/
├── app/                  Flask application
│   ├── __init__.py       app factory
│   ├── models.py         User, Equipment, Loan + business rules
│   ├── routes.py         auth, student, lab-assistant pages, JSON API, CSV export
│   ├── metrics.py        Prometheus business metrics
│   ├── seed.py           demo data
│   ├── templates/        login, register, student/, admin/
│   └── static/style.css
├── tests/                pytest suite (test_app.py, conftest.py)
├── monitoring/           prometheus/prometheus.yml, grafana/ dashboard + provisioning
├── jenkins/              Jenkins image, plugins.txt, casc.yaml (configuration as
code)
├── jira/                 LabLend_Jira_Import.csv (5 epics, 18 stories/tasks)
├── docs/                 LAB_GUIDE.md, screenshots/, evidence/
├── Dockerfile            multi-stage image (test + production)
├── docker-compose.yml    web, db, prometheus, node-exporter, grafana
├── Jenkinsfile           CI/CD pipeline
├── requirements.txt / requirements-dev.txt
└── wsgi.py
```

Figure 4.3: LabLend project repository structure

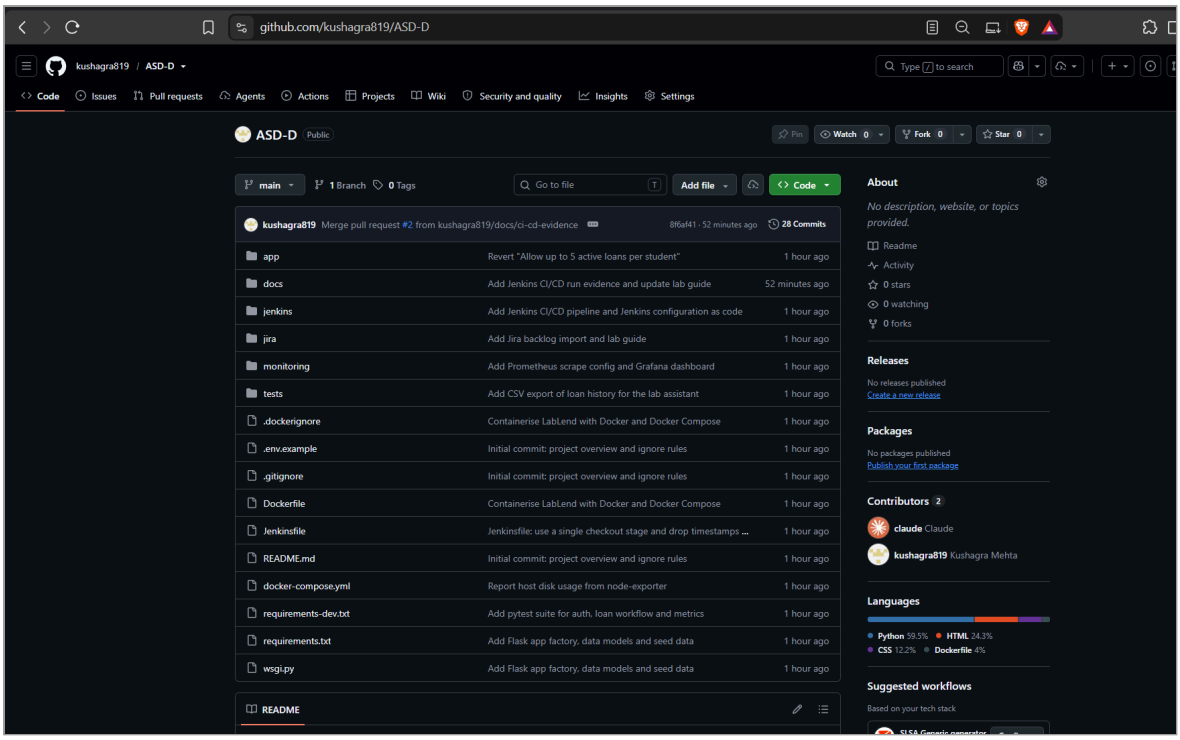


Figure 4.4: LabLend GitHub repository on the main branch

5. DevOps Implementation

5.1 Version Control Using Git and GitHub

Git was used as the distributed version-control system for LabLend. The project repository maintains the Flask backend, HTML templates, static files, Docker configuration, Jenkinsfile, tests, Jira backlog and monitoring configuration.

Changes were staged and committed using Git, while GitHub was used as the remote repository (`https://github.com/kushagra819/ASD-D`). Each part of the project was developed on its own branch (`feature/app-core`, `feature/tests`, `feature/docker`, `feature/monitoring`, `feature/jenkins`, `docs/agile-and-guide`, `fix/prometheus-volume`, `fix/node-exporter-disk`) and merged into the main line with `--no-ff` so that every merge is visible in the history. `git log --oneline --graph` was used to inspect commit history.

A new feature, CSV export of the loan register for the lab assistant, was developed on the branch `feature/loan-csv-export` and merged into `main` through Pull Request #1. To demonstrate rollback, a commit that raised the active-loan limit from 3 to 5 was pushed to `main`; Jenkins build #3 failed on the unit test that enforces the agreed limit, and the change was rolled back with `git revert`, after which build #4 passed and redeployed the application. The Jenkins run evidence was later added through Pull Request #2 (`docs/ci-cd-evidence` → `main`).

LO2: “Use Git effectively for version control, source code management, and collaborative team development.”

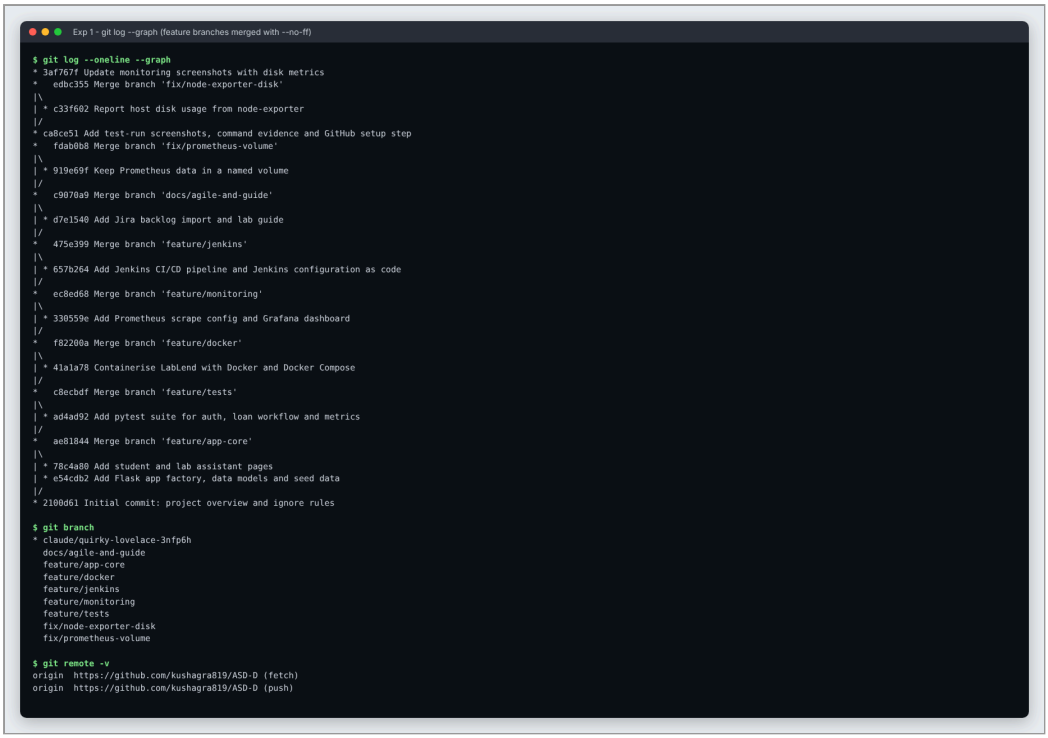


Figure 5.1: Git log showing feature branches merged into the main line

```
$ git branch
* claude/quirky-lovelace-3nfp6h
  docs/agile-and-guide
  feature/app-core
  feature/docker
  feature/jenkins
  feature/monitoring
  feature/tests
  fix/node-exporter-disk
  fix/prometheus-volume

$ git remote -v
origin  https://github.com/kushagra819/ASD-D (fetch)
origin  https://github.com/kushagra819/ASD-D (push)
```

Figure 5.2: Git branches and remote repository

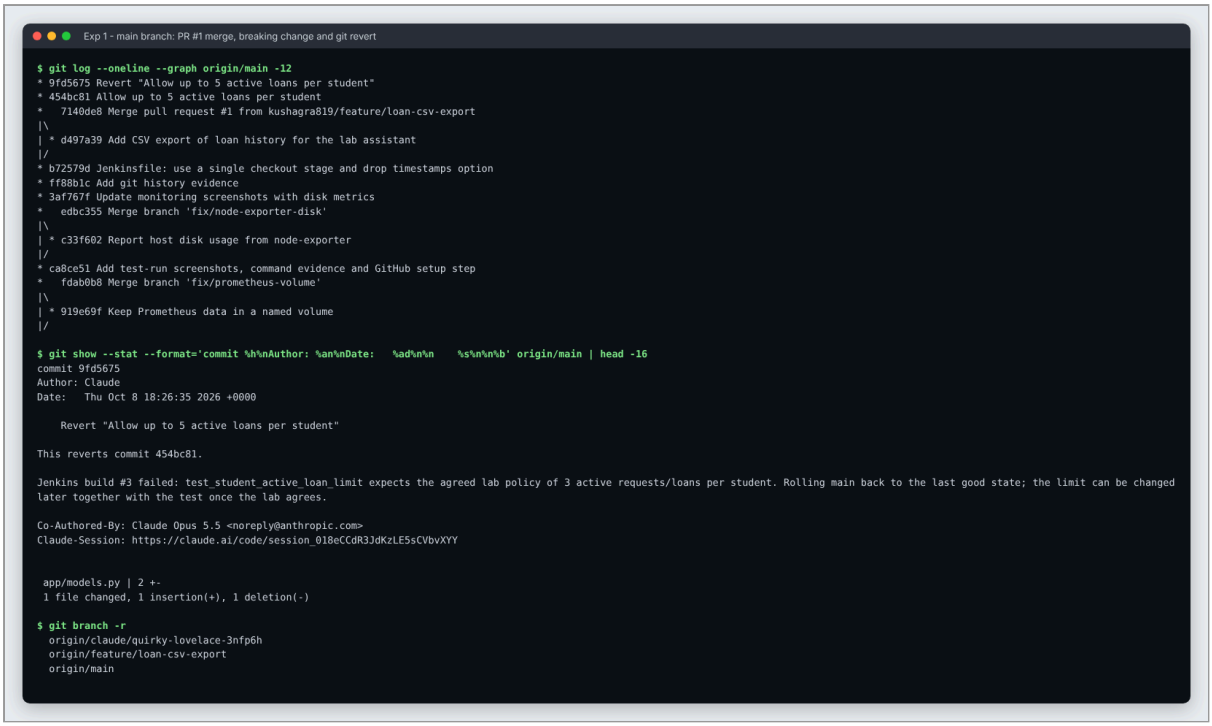


Figure 5.3: Pull Request #1 merge, breaking change and git revert on main

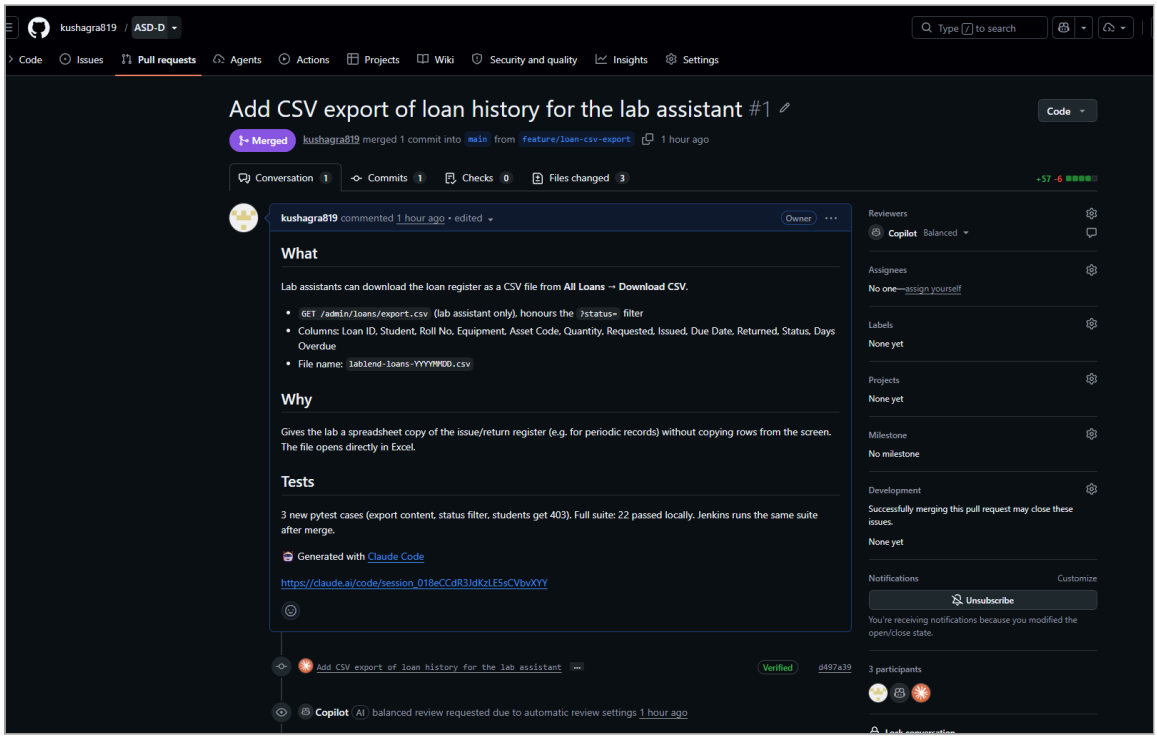


Figure 5.4: GitHub Pull Request #1 (feature/loan-csv-export → main) merged

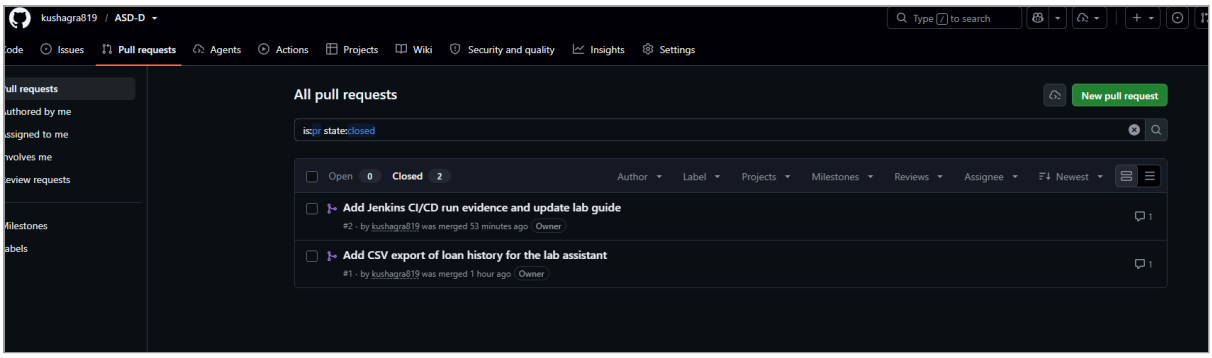


Figure 5.5: Pull requests merged into main on GitHub

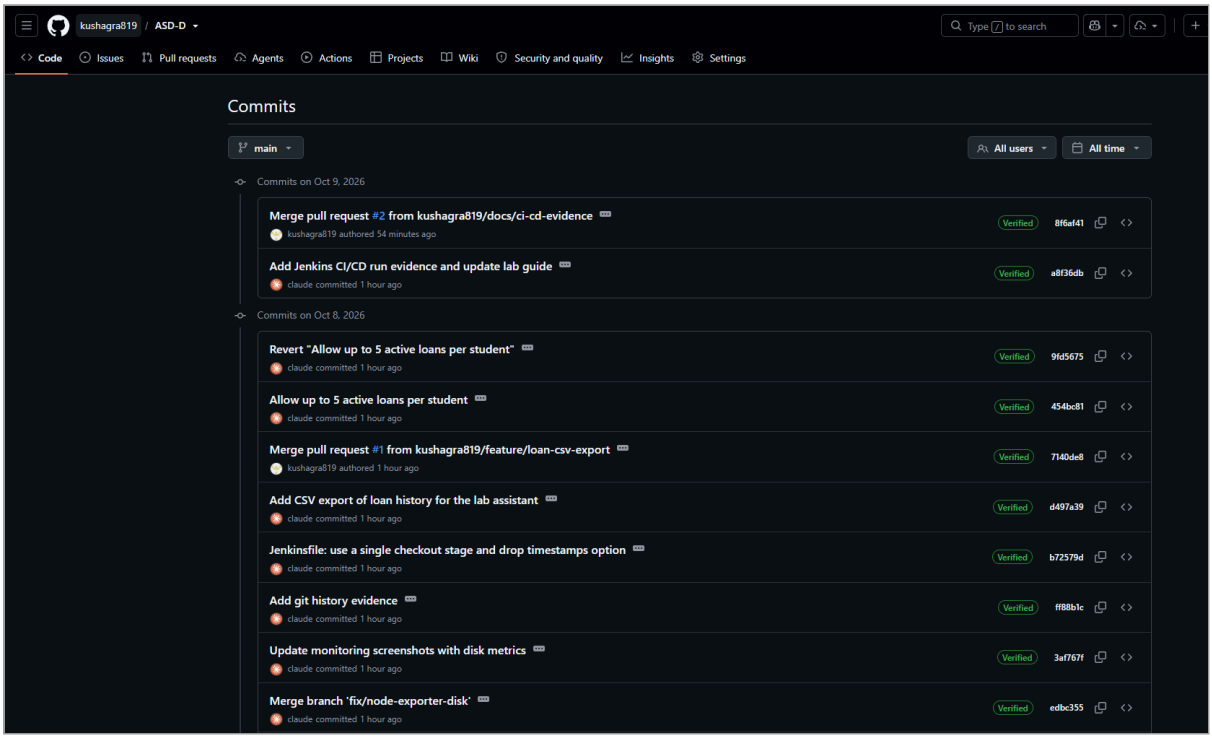


Figure 5.6: GitHub commit history on main: PR merges, breaking change and revert

5.2 Containerization Using Docker

Docker packages an application together with its dependencies, libraries, runtime and configuration into a consistent environment. A Docker image acts as the template, while a container is a running instance created from that image.

For LabLend, a multi-stage Dockerfile was created using the `python:3.12-slim` base image. The `base` stage sets the working directory, copies `requirements.txt`, installs the required packages and copies the source code. The `test` stage adds pytest and the tests and is used by Jenkins to run the unit tests. The `production` stage runs as a non-root user, exposes port 5000, defines a HEALTHCHECK on /health and starts the application with Gunicorn.

The image was built using:

```
docker build -t lablend .
```

A container was created from the image and port 5000 of the container was mapped to port 5001 of the host. `docker images`, `docker ps`, `docker logs`, `docker stop`, `docker start`, `docker inspect` and `docker rm` were used for container management and troubleshooting.

LO4: “Containerize, orchestrate, and deploy applications using Docker, Docker Compose, Kubernetes, and cloud infrastructure tools.”

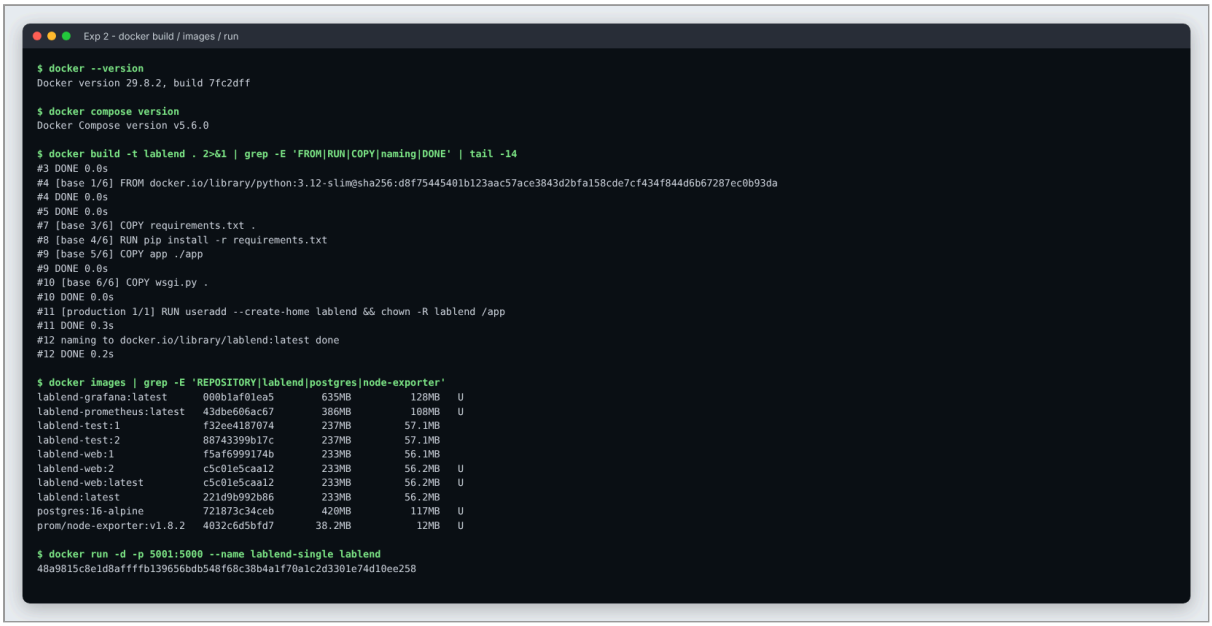


Figure 5.7: Docker image build and images list

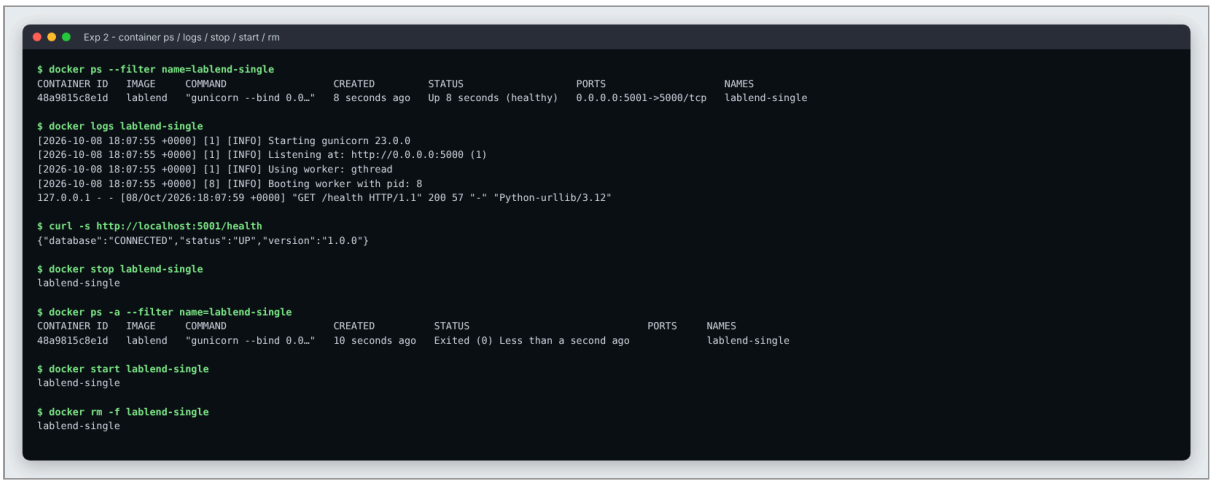


Figure 5.8: Docker container lifecycle and logs

```
$ docker ps --filter name=lablend-single
CONTAINER ID   IMAGE     COMMAND                  CREATED         STATUS
PORTS                     NAMES
48a9815c8e1d   lablend   "gunicorn --bind 0.0…"   8 seconds ago   Up 8 seconds
(healthy)   0.0.0.0:5001->5000/tcp   lablend-single

$ docker logs lablend-single
[2026-10-08 18:07:55 +0000] [1] [INFO] Starting gunicorn 23.0.0
[2026-10-08 18:07:55 +0000] [1] [INFO] Listening at: http://0.0.0.0:5000 (1)
[2026-10-08 18:07:55 +0000] [1] [INFO] Using worker: gthread
[2026-10-08 18:07:55 +0000] [8] [INFO] Booting worker with pid: 8
127.0.0.1 - - [08/Oct/2026:18:07:59 +0000] "GET /health HTTP/1.1" 200 57 "-" "Python-
urllib/3.12"

$ curl -s http://localhost:5001/health
{"database":"CONNECTED","status":"UP","version":"1.0.0"}
```

Figure 5.9: LabLend Docker container execution (health check from the container)

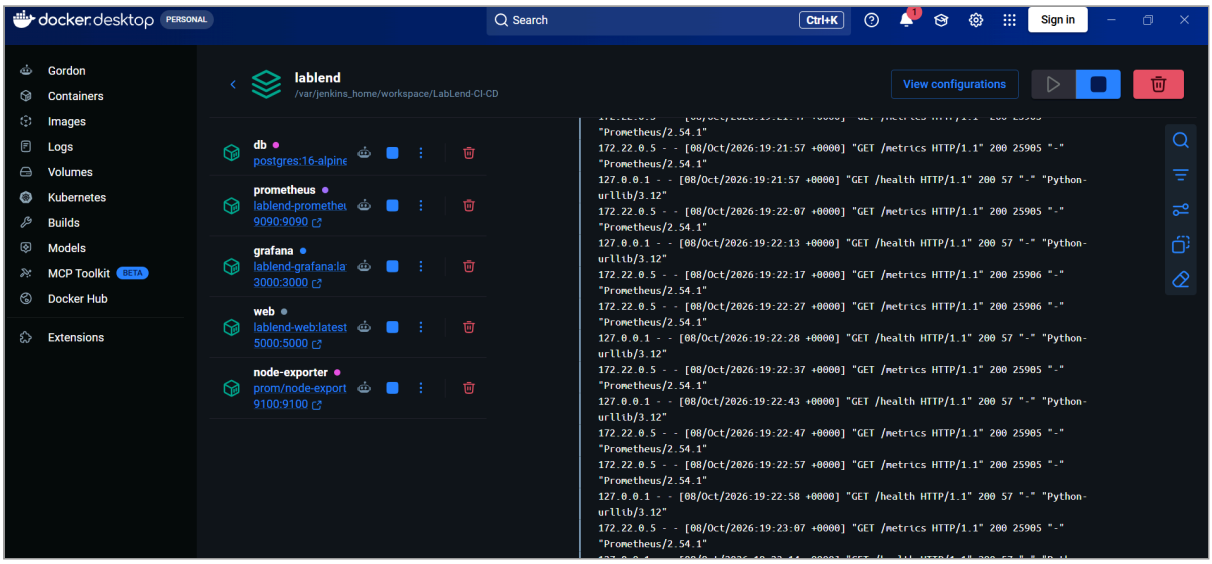


Figure 5.10: Docker Desktop: the lablend Compose stack (db, prometheus, grafana, web, node-exporter) with live web logs

5.3 Multi-Service Deployment Using Docker Compose

Docker Compose extends Docker to multi-service applications by allowing related services to be declared in a single YAML configuration file. Each service runs in its own container while communicating with the other services.

LabLend requires both an application server and a database, and the monitoring stack runs alongside them, so `docker-compose.yml` was created with five services: `web` runs the Flask application, `db` uses the `postgres:16-alpine` image, and `prometheus`, `node-exporter` and `grafana` provide monitoring. The Flask container communicates with PostgreSQL using the Compose service name `db` and the `DATABASE_URL` environment variable.

A named Docker volume, `pgdata`, provides persistent database storage, while a PostgreSQL health check (`pg_isready`) ensures that the Flask service waits for the database service to become healthy before starting. Commands such as `docker compose config`, `docker`

`compose up -d --build`, `docker compose ps` and `docker compose logs` were used for validation and troubleshooting.

LO4: “Containerize, orchestrate, and deploy applications using Docker, Docker Compose, Kubernetes, and cloud infrastructure tools.”

```
  db:
    image: postgres:16-alpine
    restart: unless-stopped
    environment:
      POSTGRES_DB: lablend
      POSTGRES_USER: lablend
      POSTGRES_PASSWORD: ${DB_PASSWORD:-lablend123}
    volumes:
      - pgdata:/var/lib/postgresql/data
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U lablend -d lablend"]
      interval: 5s
      timeout: 5s
      retries: 10

  web:
    build:
      context: .
      target: production
    image: lablend-web:latest
    restart: unless-stopped
    ports:
      - "5000:5000"
    environment:
      DATABASE_URL: postgresql://lablend:${DB_PASSWORD:-lablend123}@db:5432/lablend
      SECRET_KEY: ${SECRET_KEY:-change-this-secret}
      ADMIN_ROLL: ${ADMIN_ROLL:-labadmin}
      ADMIN_PASSWORD: ${ADMIN_PASSWORD:-admin123}
      SEED_DEMO_DATA: "true"
      APP_VERSION: ${APP_VERSION:-1.0.0}
    depends_on:
      db:
        condition: service_healthy
```

Figure 5.11: Docker Compose configuration (web and db services)

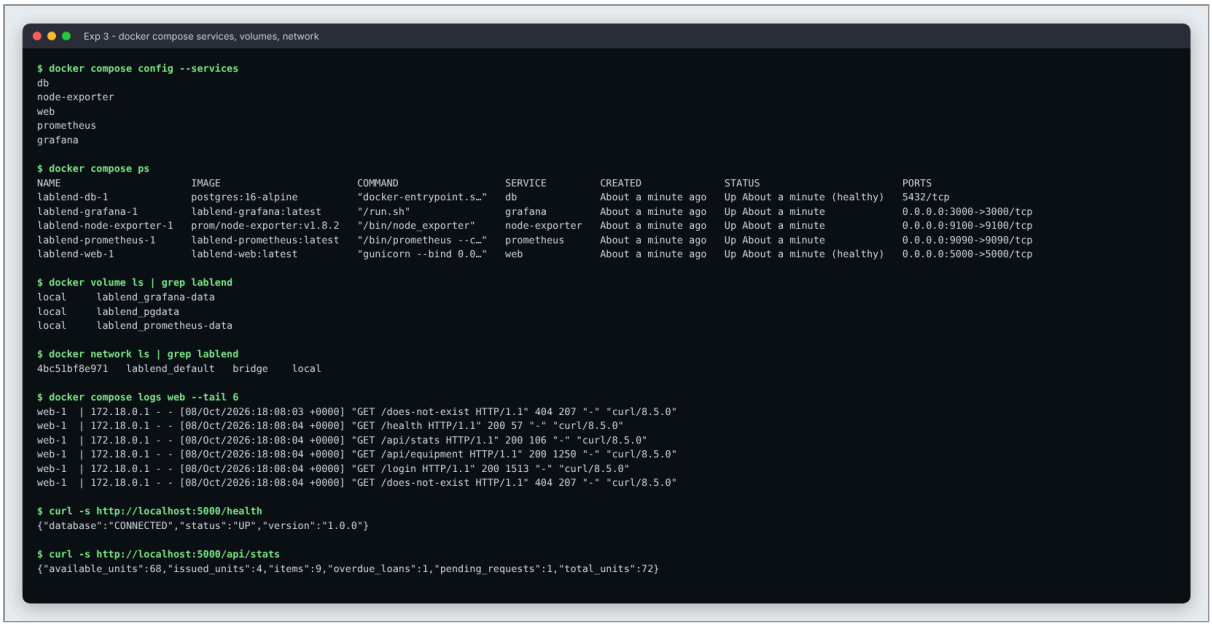


Figure 5.12: Docker Compose services, volumes, network and logs

```
$ docker compose config --services
db
node-exporter
web
prometheus
grafana

$ docker compose ps
NAME                      IMAGE                          COMMAND                  SERVICE
CREATED              STATUS                        PORTS
lablend-db-1              postgres:16-alpine             "docker-entrypoint.s…"   db
About a minute ago   Up About a minute (healthy)   5432/tcp
lablend-grafana-1         lablend-grafana:latest         "/run.sh"                grafana
About a minute ago   Up About a minute             0.0.0.0:3000->3000/tcp
lablend-node-exporter-1   prom/node-exporter:v1.8.2      "/bin/node_exporter"     node-
exporter   About a minute ago   Up About a minute             0.0.0.0:9100->9100/tcp
lablend-prometheus-1      lablend-prometheus:latest      "/bin/prometheus --c…"
prometheus      About a minute ago   Up About a minute             0.0.0.0:9090-
>9090/tcp
lablend-web-1             lablend-web:latest             "gunicorn --bind 0.0…"   web
About a minute ago   Up About a minute (healthy)   0.0.0.0:5000->5000/tcp

$ docker volume ls | grep lablend
```

Figure 5.13: Docker Compose services/status

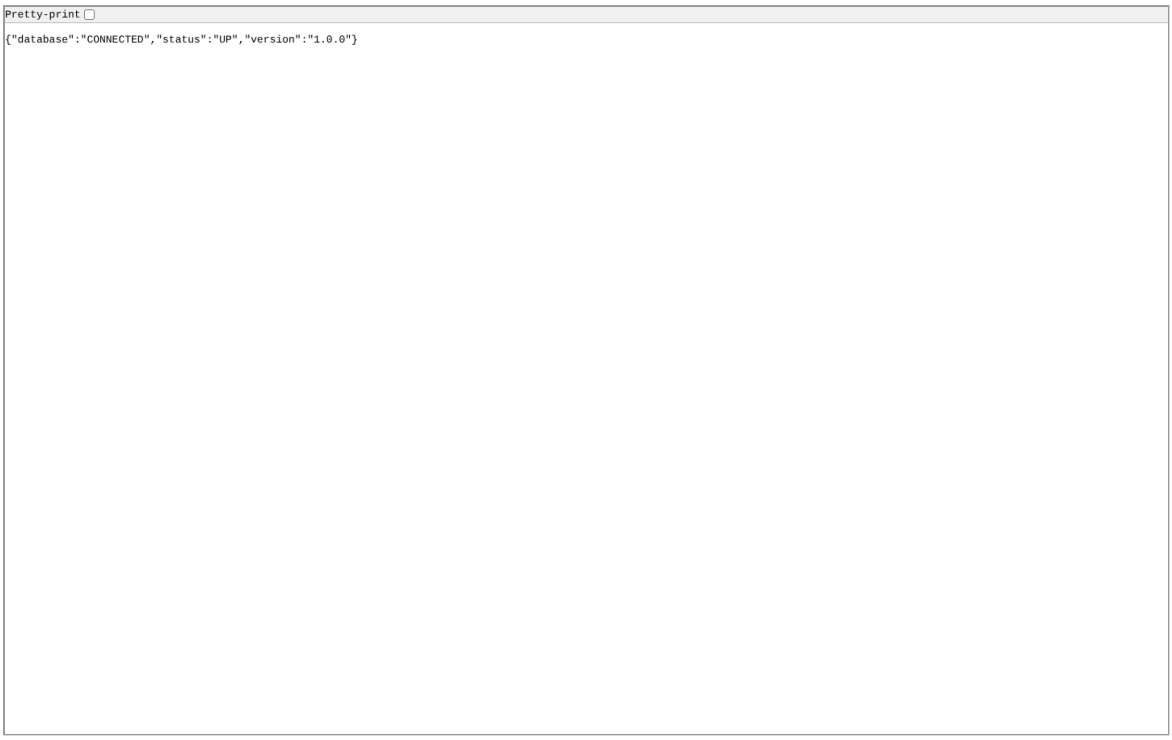


Figure 5.14: Running multi-service environment: /health reports the database CONNECTED

5.4 Continuous Integration Using Jenkins

Continuous Integration is a development practice in which developers frequently integrate changes into a shared repository and automated processes build and test those changes. Jenkins is an automation server used for CI/CD activities such as building applications, executing tests and performing deployment operations.

Jenkins was run in a Docker container, and a pipeline job named **LabLend-CI-CD** was connected to the GitHub repository `https://github.com/kushagra819/ASD-D.git`, branch `*/main`. The job is created automatically through Jenkins Configuration as Code (`jenkins/casc.yaml`), and the `Jenkinsfile` is stored with the project source code. The CI part of the pipeline checks out the source code, builds the test image and runs the automated tests using pytest inside it, publishing a JUnit test report.

GitHub and Jenkins were connected through SCM polling: Jenkins checks the repository every 2 minutes, so new pushes and merged pull requests on `main` automatically start the pipeline (build #2 was started by the merge of Pull Request #1). The job also accepts GitHub webhooks, which can be exposed through an ngrok tunnel when Jenkins runs on a laptop.

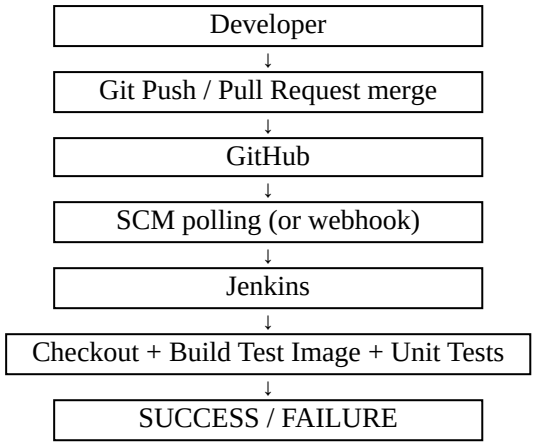


Figure 5.15: Continuous Integration flow from GitHub to Jenkins

LO3: "Build and manage automated CI/CD pipelines for application development, testing, packaging, and deployment."

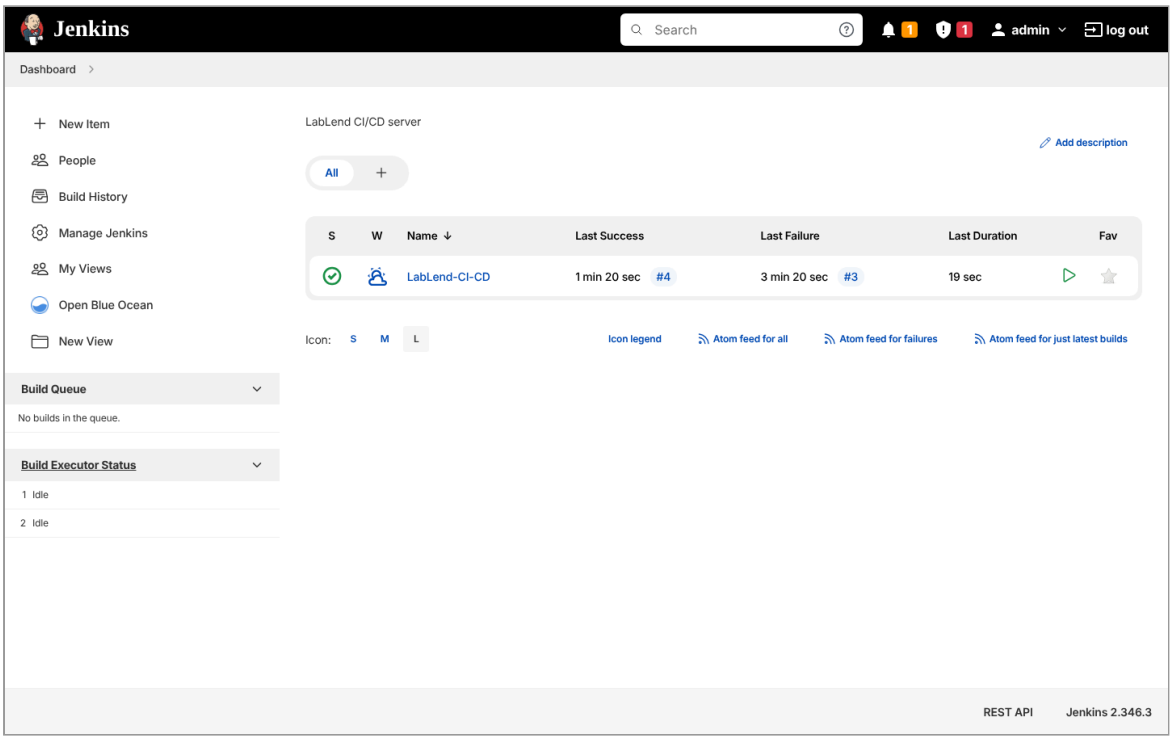


Figure 5.16: Jenkins dashboard with the LabLend-CI-CD job

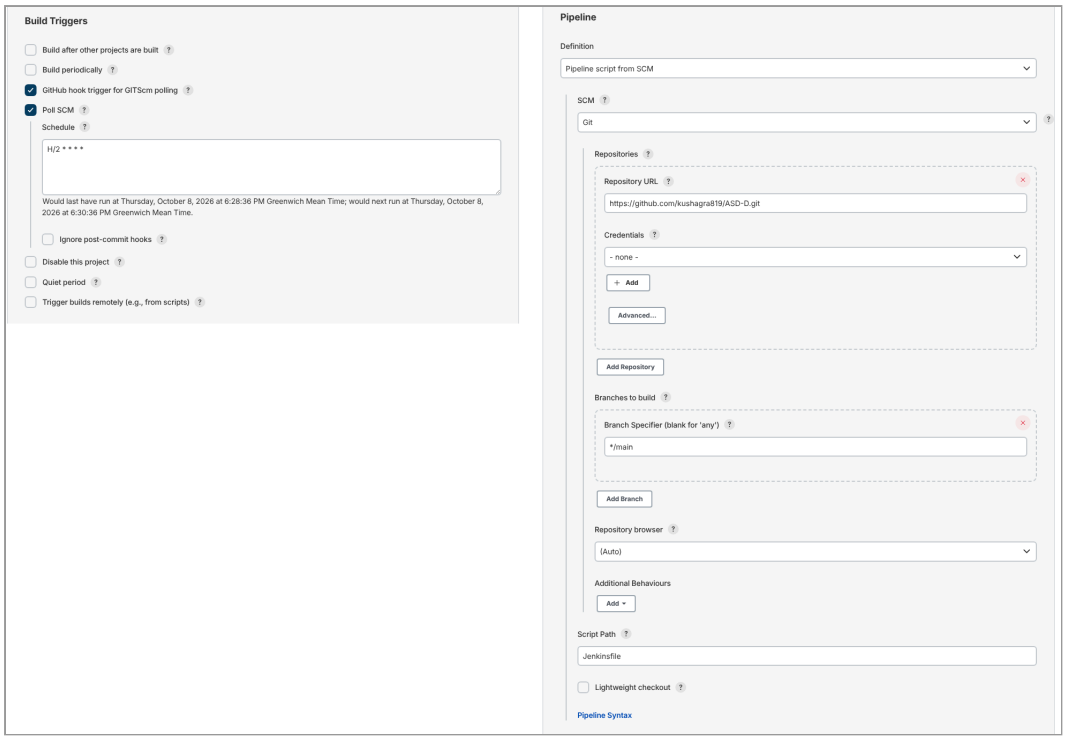


Figure 5.17: Jenkins pipeline configuration: GitHub repository, */main, SCM polling, Jenkinsfile

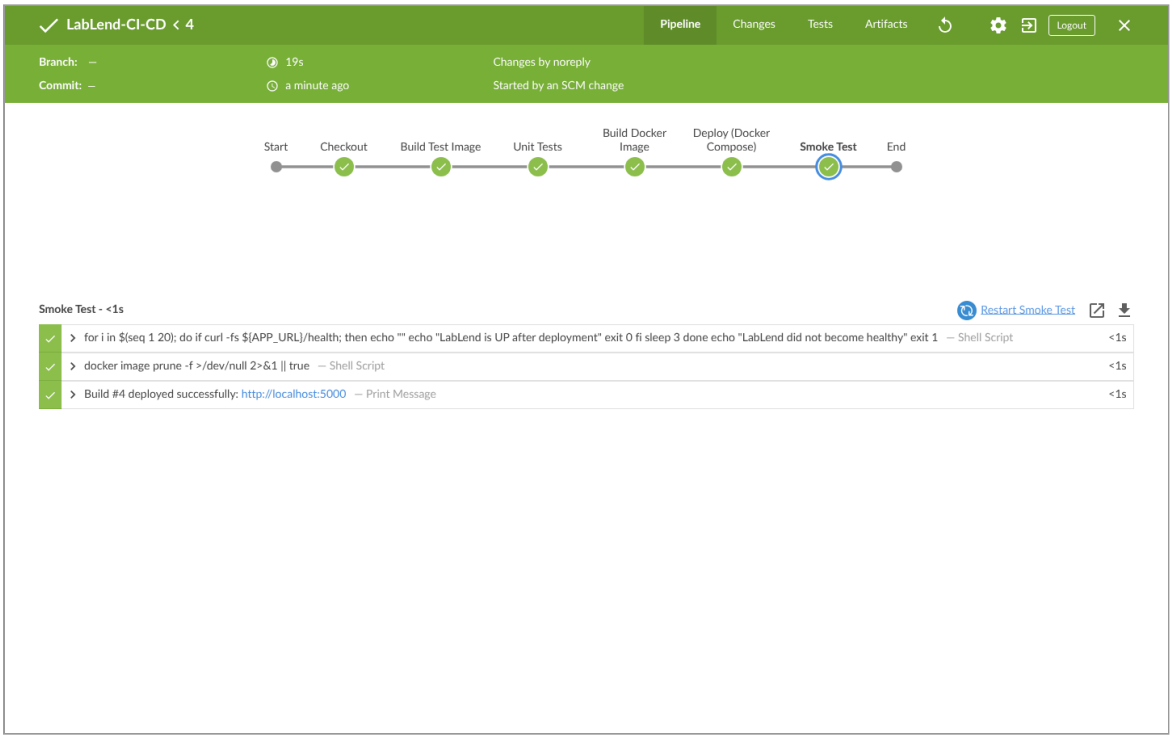


Figure 5.18: Jenkins build execution: build #4 with all six stages successful

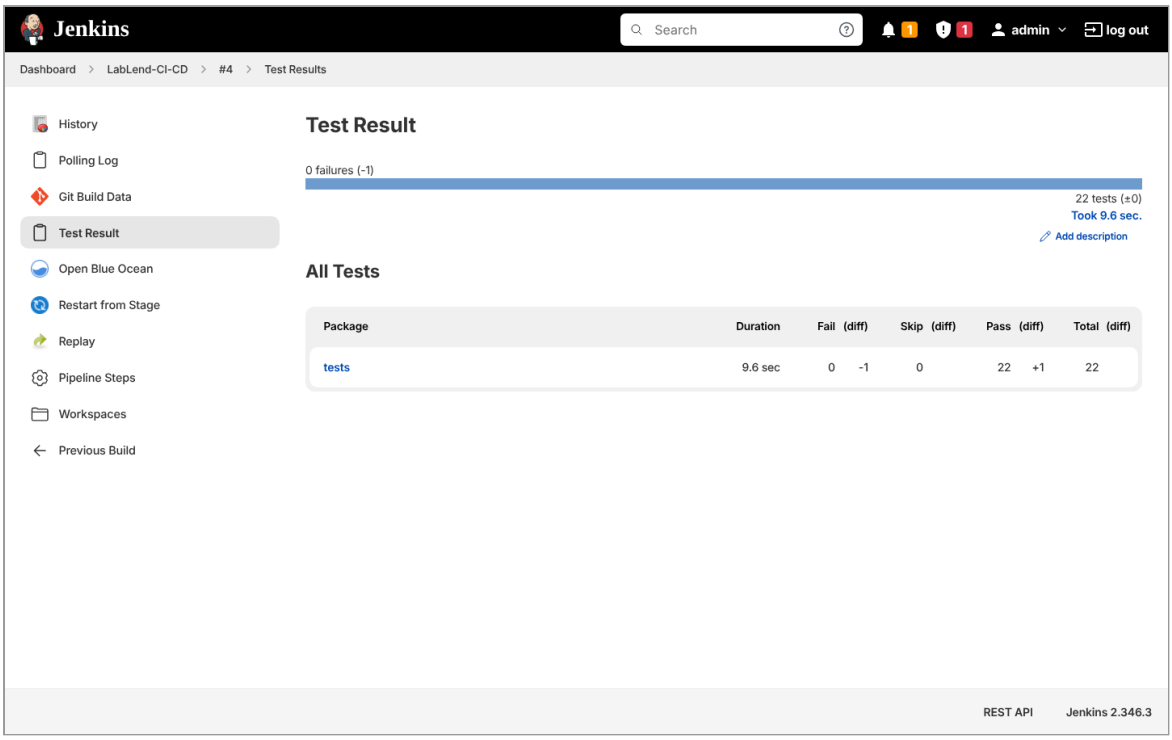


Figure 5.19: Jenkins pipeline result: JUnit test report with 22 tests passed

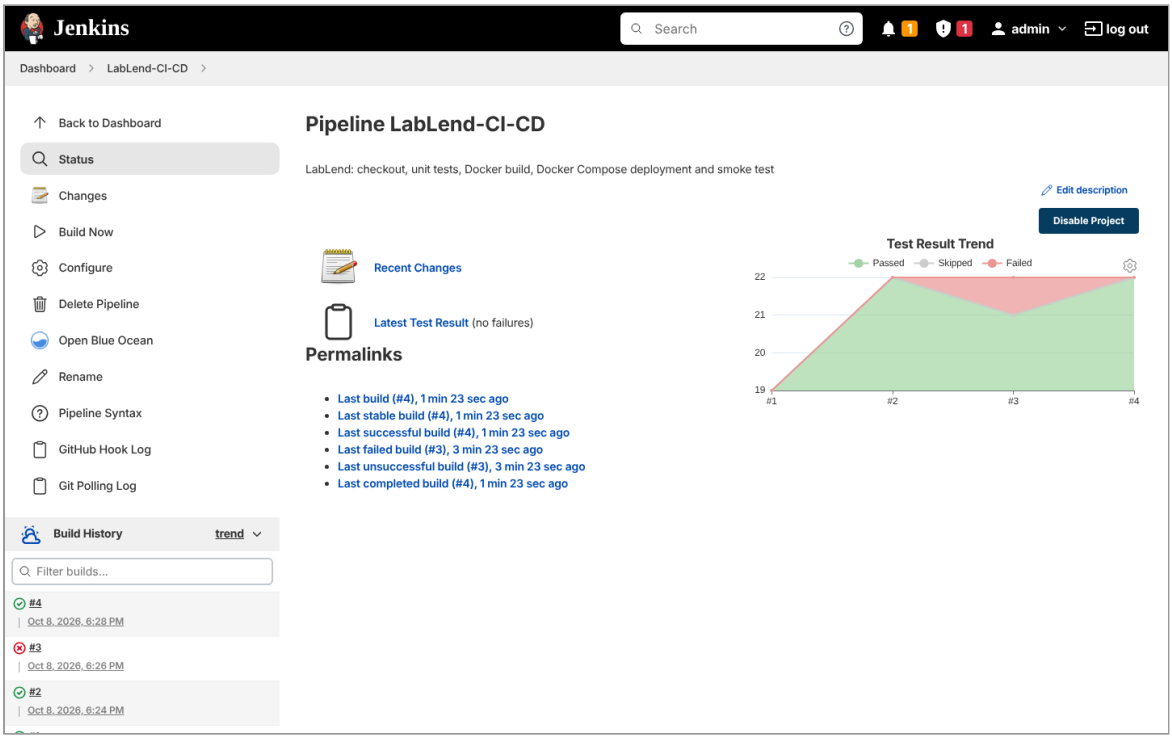


Figure 5.20: Jenkins build history #1–#4

5.5 Automated Deployment Using GitHub and Jenkins

This extends Continuous Integration toward Continuous Deployment. The Jenkins pipeline created in Experiment 4 was extended to perform Docker image building, deployment and a smoke test. The resulting workflow became:

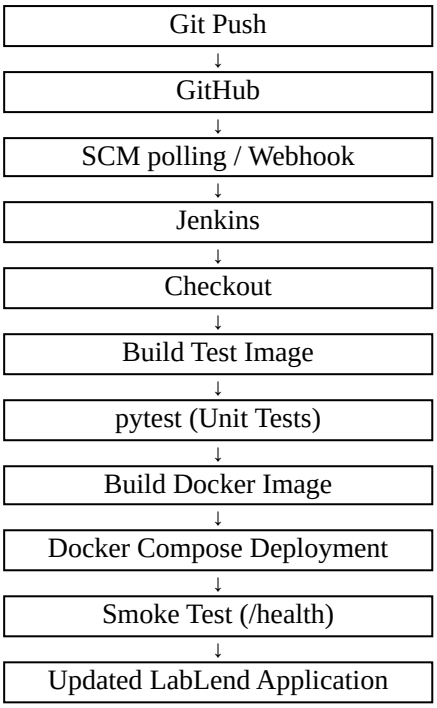


Figure 5.21: Automated deployment pipeline

The pipeline first retrieves the latest source code and builds the test image. It then runs the LabLend automated tests. Only after the testing stage succeeds does Jenkins proceed to build the production Docker image (tagged with the build number and `latest`), deploy the application using Docker Compose with `APP_VERSION=1.0.<build number>`, and check that `/health` responds. This makes deployment part of the automated pipeline instead of requiring the developer to repeat the complete sequence manually.

Four builds were run against `main`:

Build	Started by	Change	Result
#1	User (Build Now)	main before the pull request	SUCCESS: 19 tests passed, deployed v1.0.1
#2	SCM change	Merge of Pull Request #1 (CSV export)	SUCCESS: 22 tests passed, deployed v1.0.2
#3	SCM change	“Allow up to 5 active loans per student”	FAILURE: 1 failed, 21 passed; Docker build and deploy skipped
#4	SCM change	Revert of the breaking change	SUCCESS: 22 tests passed, deployed v1.0.4

Table 5.1: Jenkins builds of LabLend-CI-CD

Build #3 shows that a faulty change is stopped at the test stage and never reaches the running application, and build #4 shows the rollback being deployed automatically.

LO3: “Build and manage automated CI/CD pipelines for application development, testing, packaging, and deployment.”

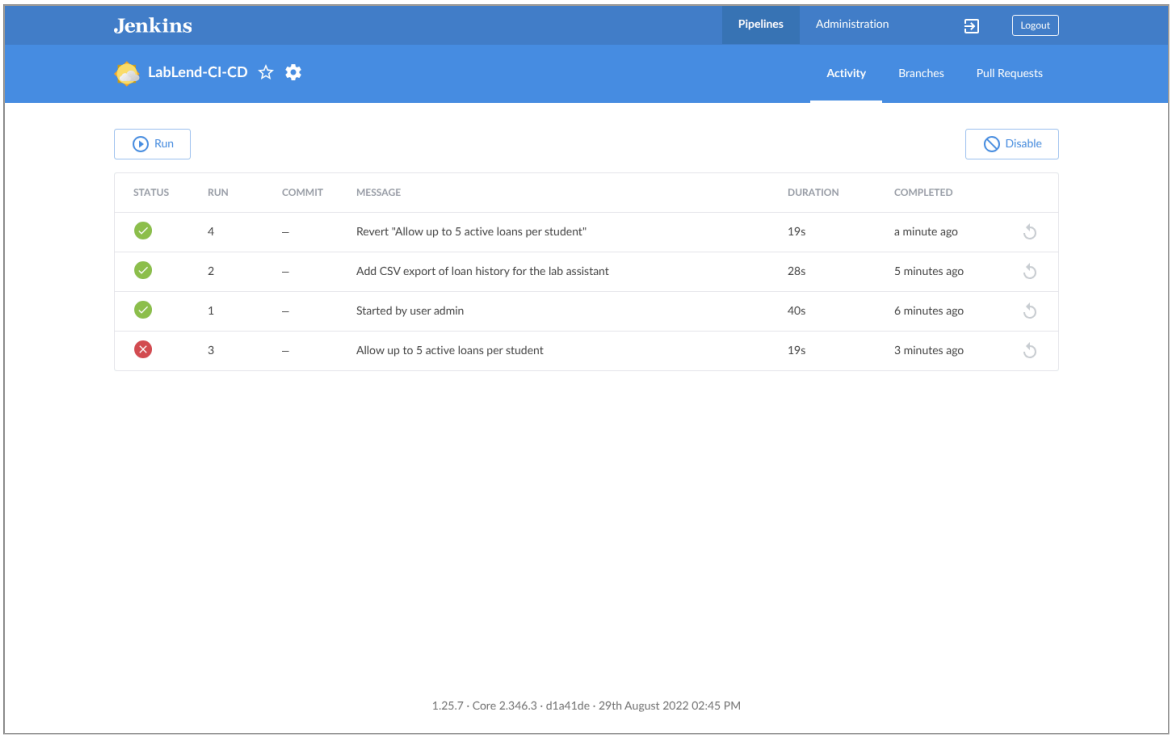


Figure 5.22: Automated deployment: Blue Ocean activity for builds #1–#4

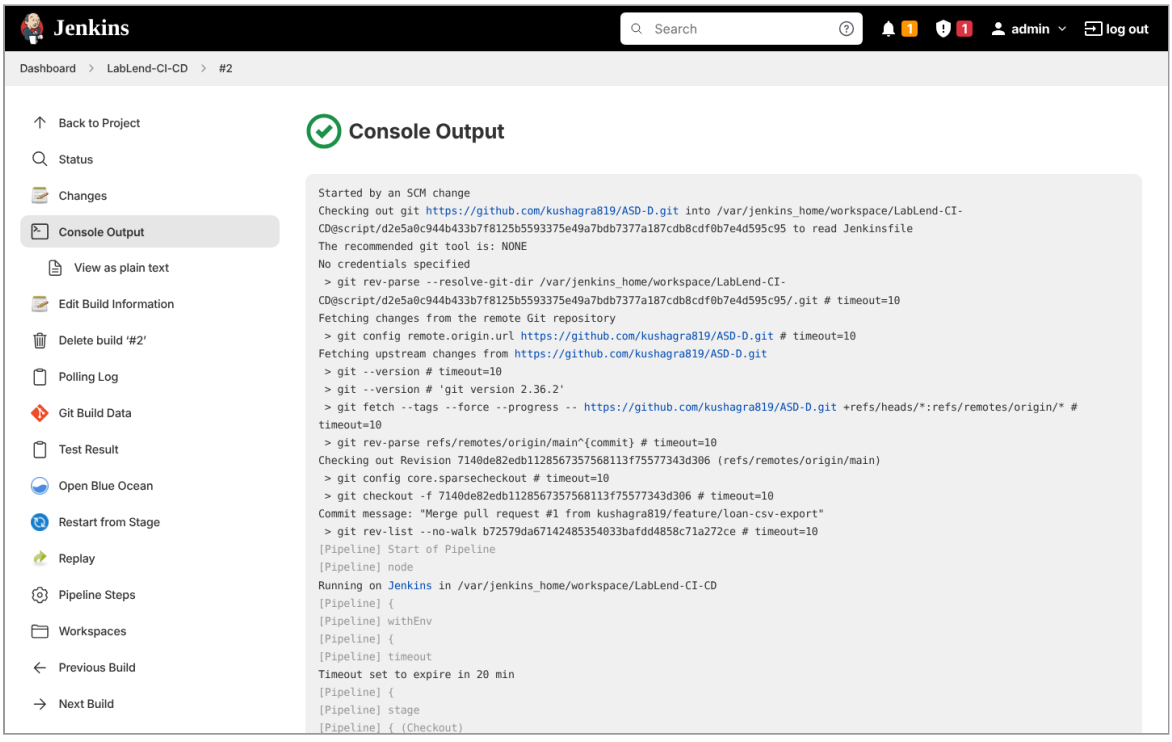


Figure 5.23: Build #2 console output: “Started by an SCM change” after the Pull Request #1 merge

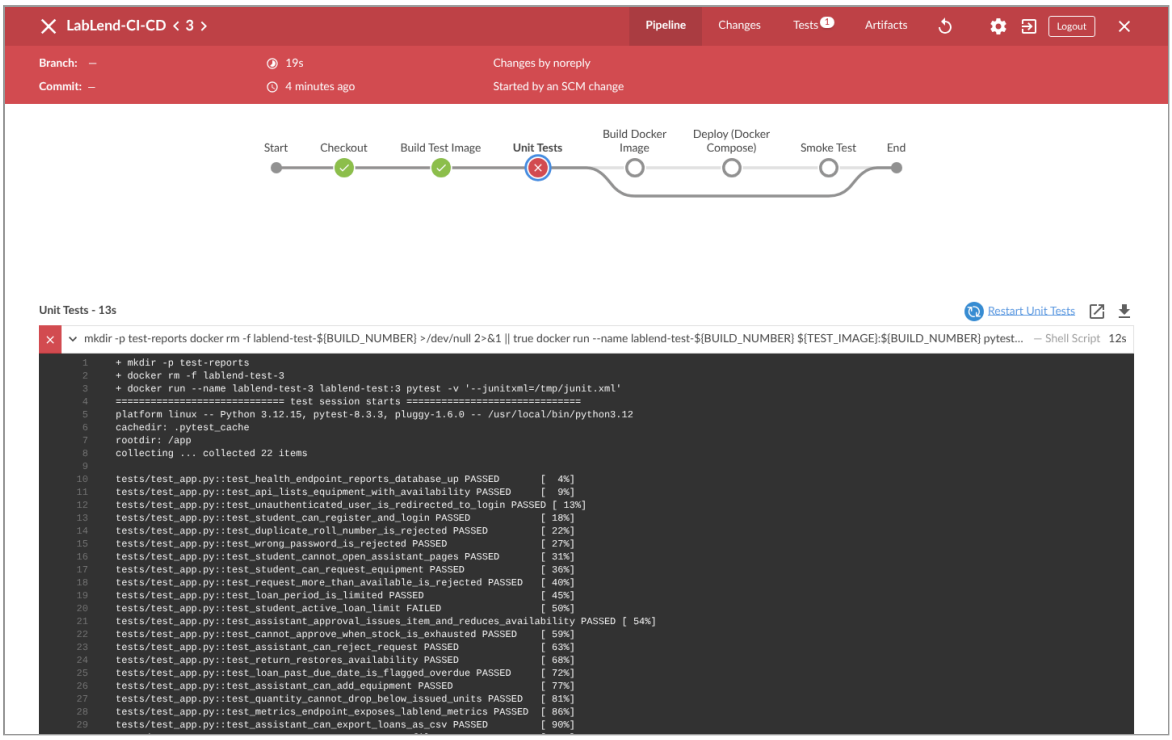


Figure 5.24: Build #3: Unit Tests failed, image build and deployment skipped

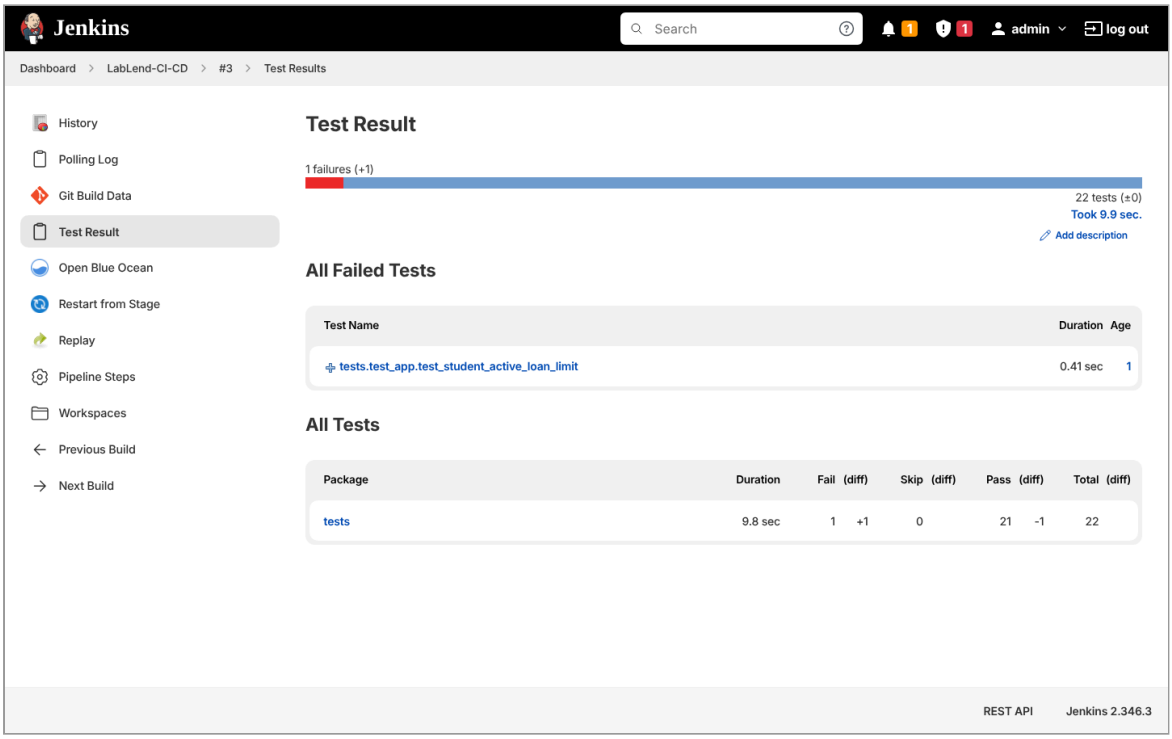


Figure 5.25: Build #3 JUnit report showing the failing test

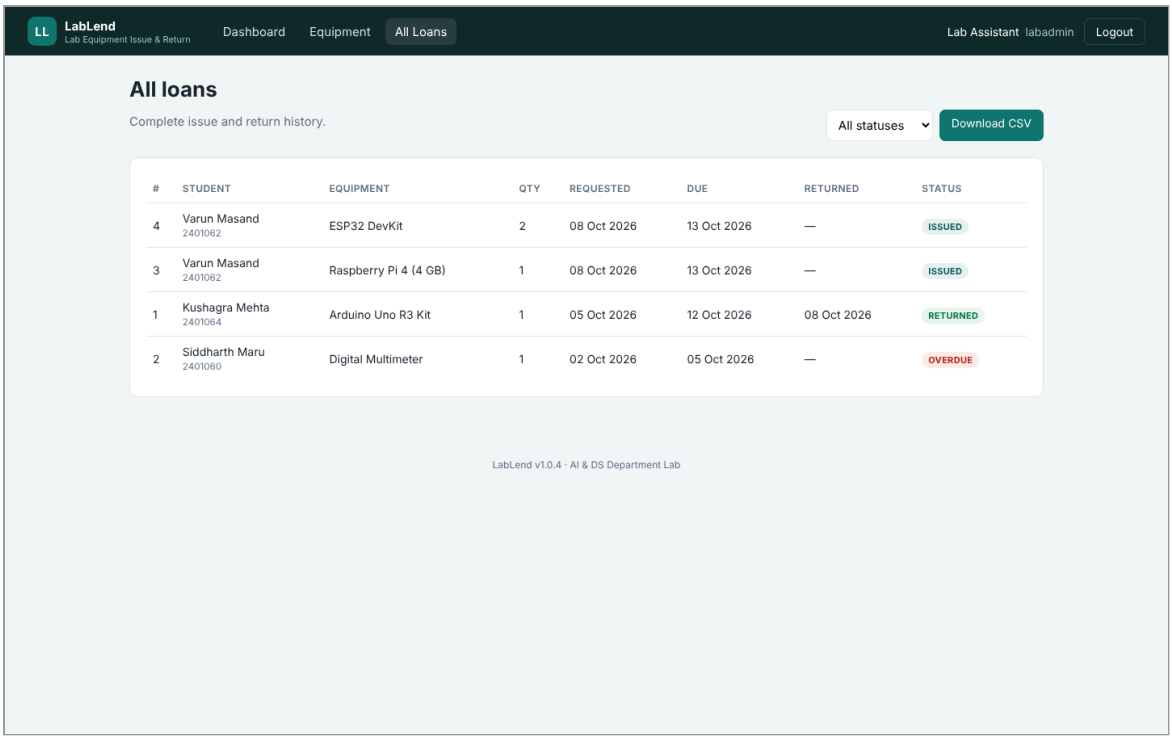


Figure 5.26: Deployed v1.0.4: All Loans page with the new Download CSV button

5.6 Agile Software Development Workflow Using Jira

Jira was used as the Agile planning and tracking tool. The experiment connects Agile development, Jira, Git/GitHub, Jenkins and Docker-related development activities into one practical workflow.

A Jira Scrum project named LabLend was created for the team at `lablend-team.atlassian.net` (project key `SCRUM`). Its backlog holds 5 epics and 19 stories and tasks (54 story points), each assigned to a team member and planned into three sprints. The epics are:

Epic	Stories / Tasks	Sprint
Equipment Catalogue	Add equipment with asset code and quantity; update stock; browse and search with live availability	Sprint 1
Issue & Return Workflow	Register/login with roll number; request equipment for a number of days; approve or reject requests; mark items as returned	Sprints 1–2
Overdue Tracking & Dashboard	Export loan register as CSV; flag overdue loans; dashboard of lab statistics; student loans and due dates	Sprints 1–2
DevOps Pipeline	Git repository and branches; Dockerfile; docker-compose.yml; pytest unit tests; Jenkins pipeline	Sprints 1–3

Epic	Stories / Tasks	Sprint
Monitoring	Expose /metrics; Prometheus and node-exporter; Grafana dashboard	Sprint 3

Table 5.2: LabLend product backlog (epics)

Each story carries a priority and a story-point estimate. Dependencies were recorded with “is blocked by” links: SCRUM-17 “Approve or reject requests” is blocked by SCRUM-15 “Request equipment for a number of days”, and SCRUM-25 “Create Jenkins pipeline” is blocked by SCRUM-24 “Write pytest unit tests”. A Scrum-style workflow was followed:

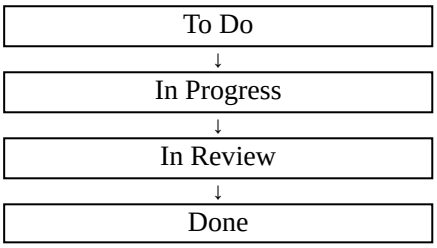


Figure 5.27: Jira issue workflow stages

The feature task “CSV export of the loan register” was implemented on a feature branch, tested and pushed to the repository, reviewed through Pull Request #1 and verified by Jenkins build #2. Its Jira story, SCRUM-16 “Export loan register as CSV”, is in the Done column of the Sprint 1 board.

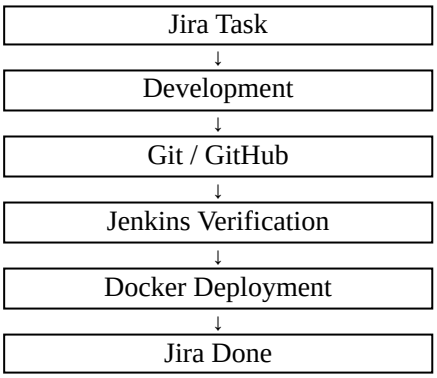


Figure 5.28: Jira task to deployment traceability

LO1: “Apply Agile methodologies and use Jira for sprint planning, task tracking, and integrating with DevOps workflows.”

The Jira figures below were generated from the live LabLend Jira project data (project `SCRUM`, read on 11 October 2026): the same work items, keys, statuses, story points, sprints, assignees and links. They are illustrative views rendered from that data, not screenshots captured from the Jira website.

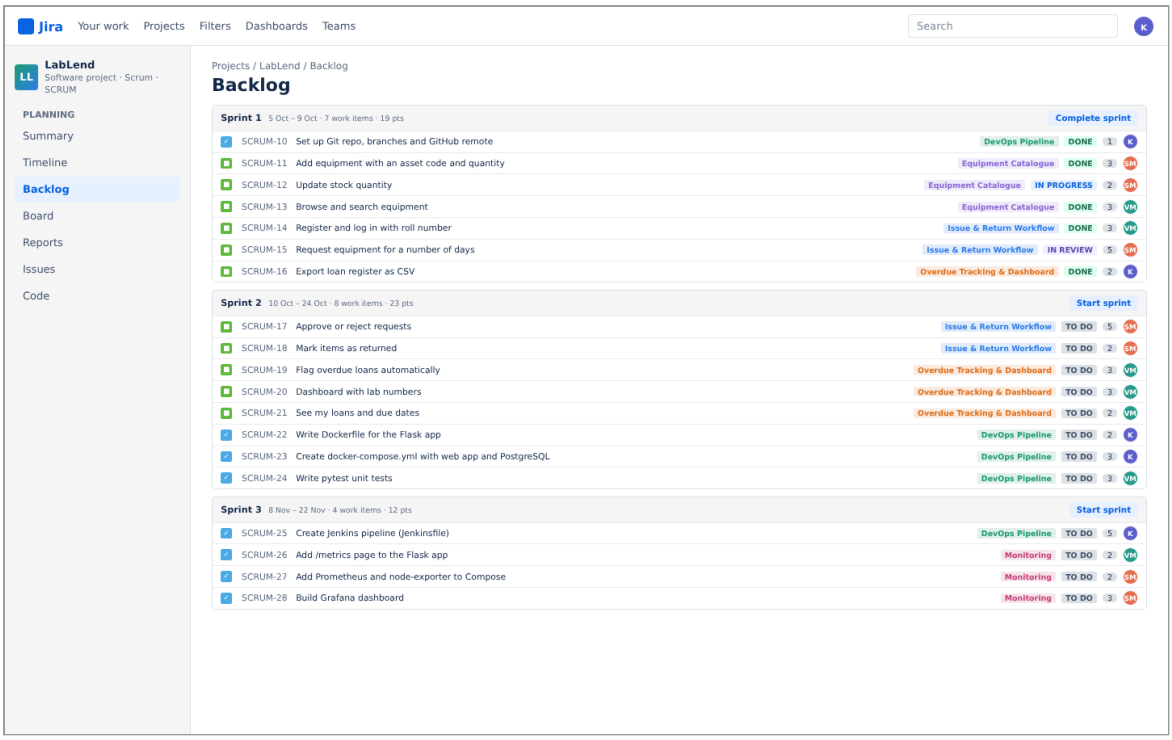


Figure 5.29: Illustrative Jira view generated from live SCRUM project data: backlog with Sprint 1 (active), Sprint 2 and Sprint 3

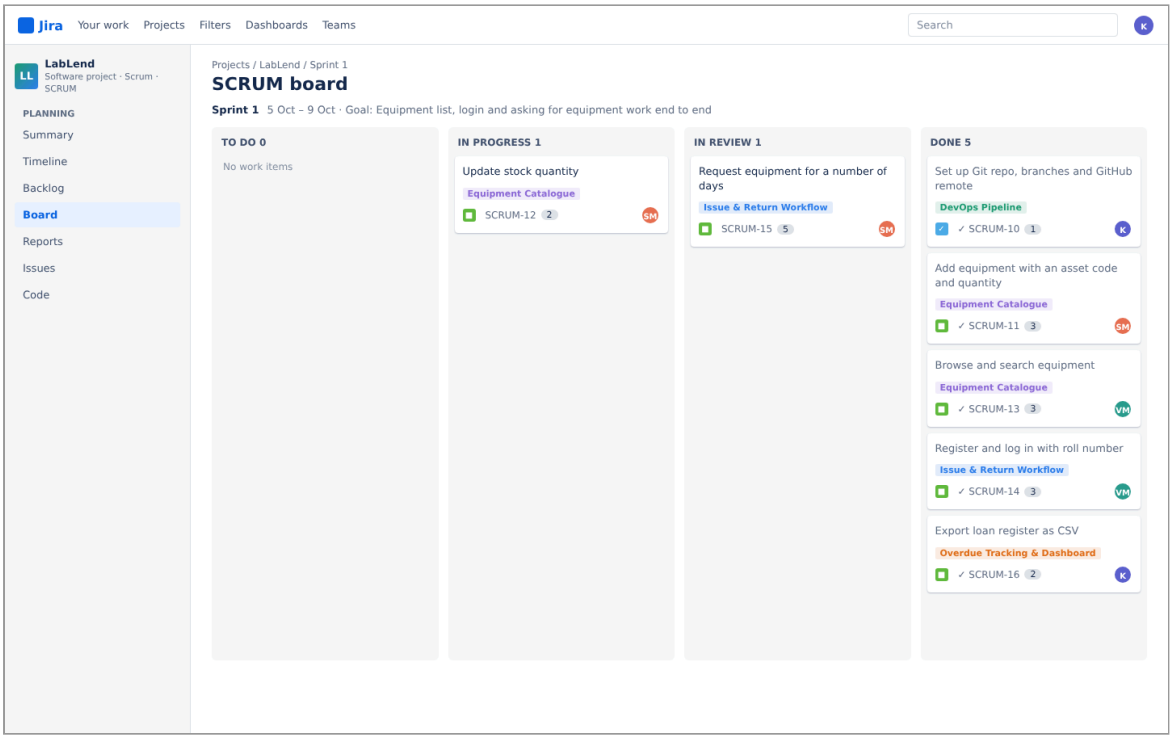


Figure 5.30: Illustrative Jira view generated from live SCRUM project data: Sprint 1 board with To Do, In Progress, In Review and Done

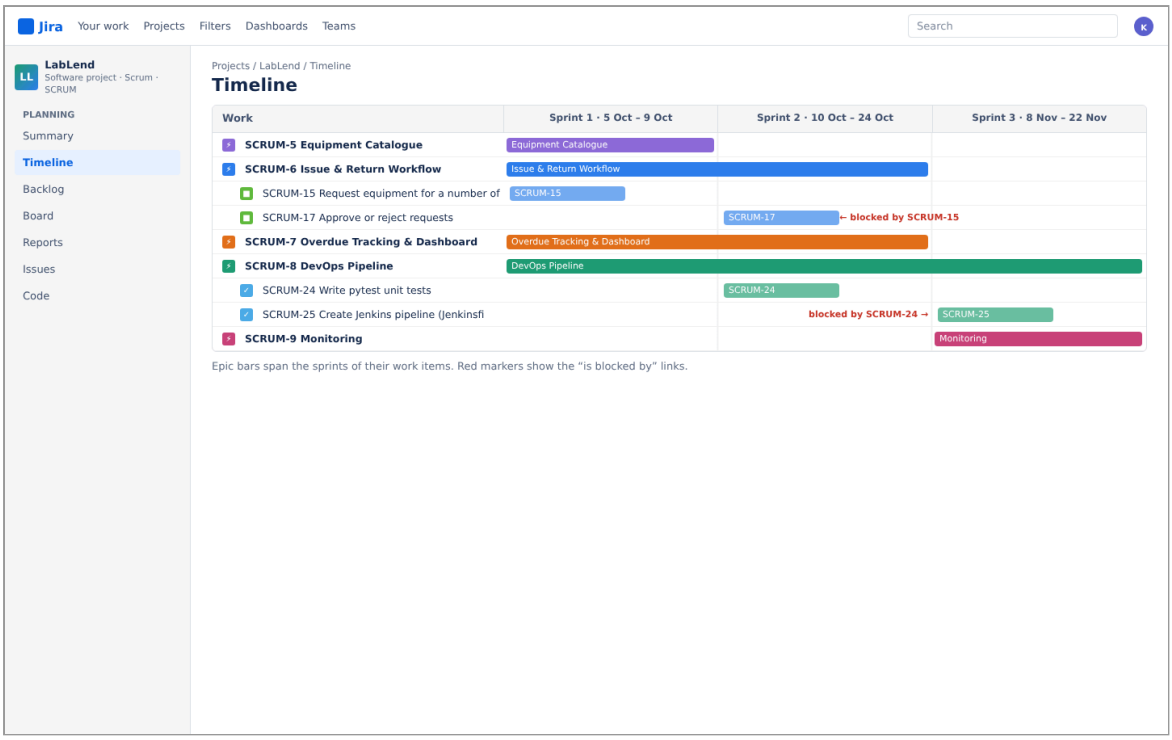


Figure 5.31: Illustrative Jira view generated from live SCRUM project data: timeline of the five epics with dependencies

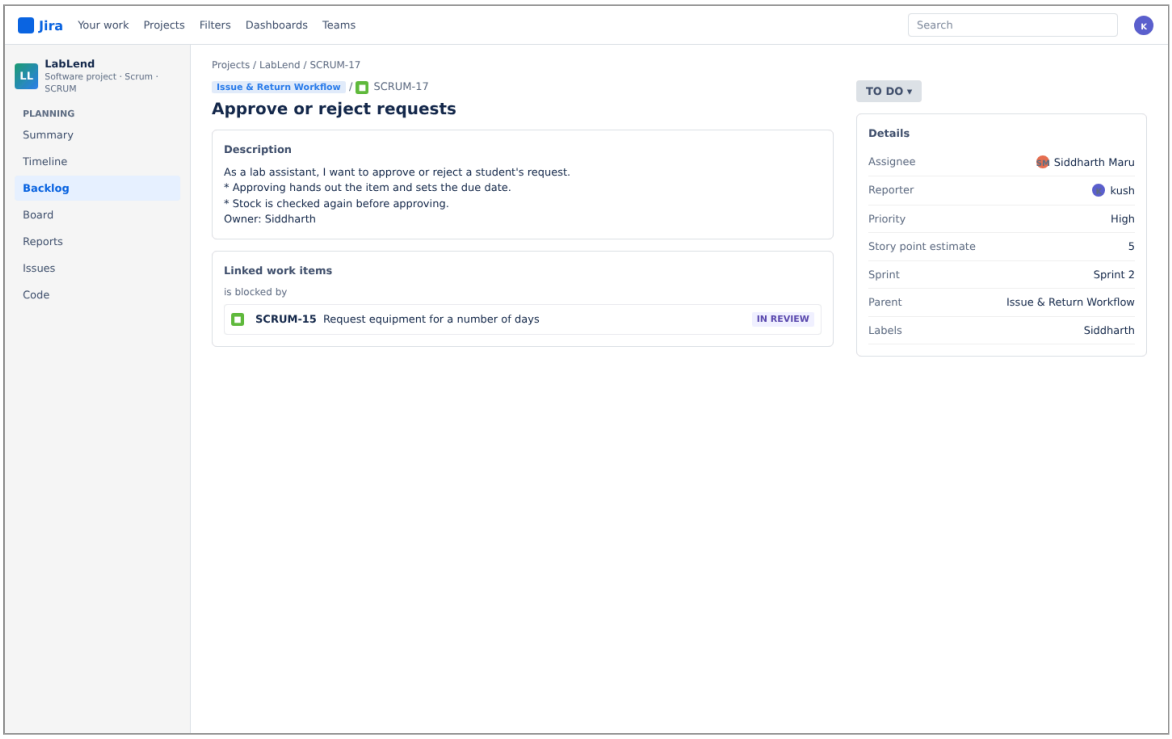


Figure 5.32: Illustrative Jira view generated from live SCRUM project data: SCRUM-17 "is blocked by" SCRUM-15

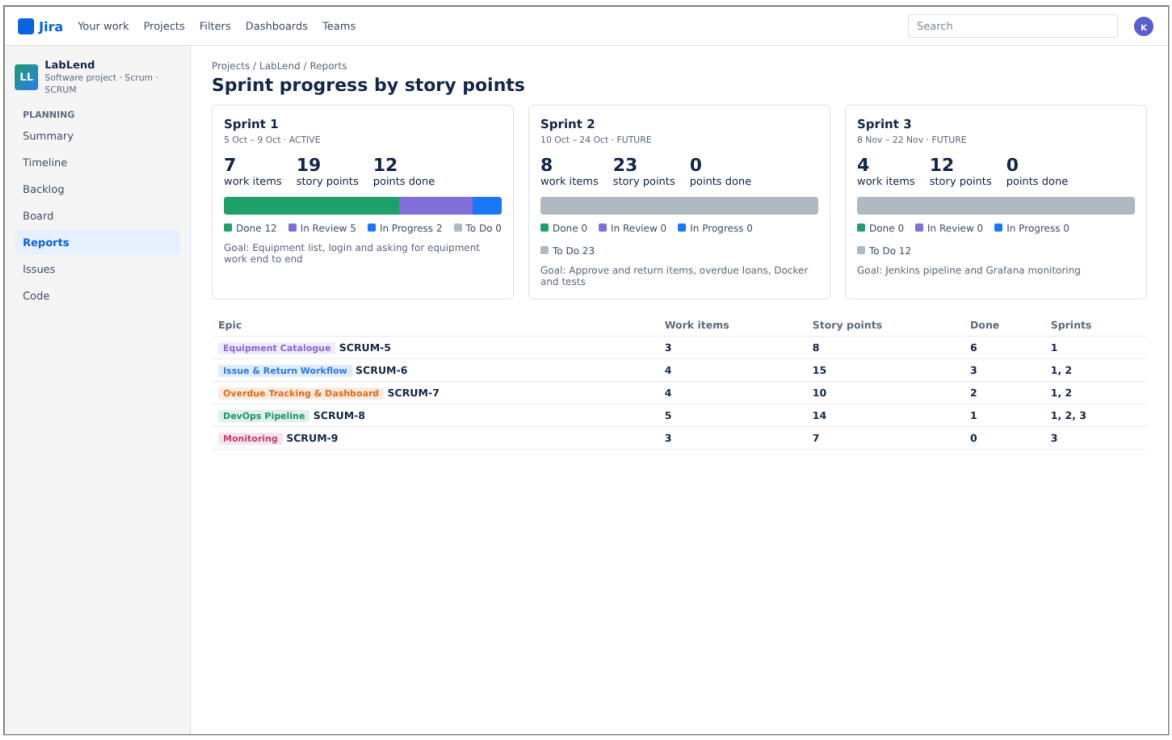


Figure 5.33: Illustrative Jira view generated from live SCRUM project data: sprint progress by story points

Epic	Work items	Story points	Points done	Sprints
SCRUM-5 Equipment Catalogue	3	8	6	1
SCRUM-6 Issue & Return Workflow	4	15	3	1, 2
SCRUM-7 Overdue Tracking & Dashboard	4	10	2	1, 2
SCRUM-8 DevOps Pipeline	5	14	1	1, 2, 3
SCRUM-9 Monitoring	3	7	0	3

Table 5.3: Epic breakdown with story points (live SCRUM project, 11 October 2026)

Issue	Link	Depends on
SCRUM-17 Approve or reject requests	is blocked by	SCRUM-15 Request equipment for a number of days
SCRUM-25 Create Jenkins pipeline (Jenkinsfile)	is blocked by	SCRUM-24 Write pytest unit tests

Table 5.4: Issue dependencies in the SCRUM project

5.7 Continuous Monitoring Using Prometheus

Continuous monitoring involves observing applications, servers, networks and infrastructure so that failures and performance-related problems can be detected after deployment. The experiment covers monitoring through Prometheus, Grafana and Nagios. Prometheus collects and stores time-series metrics, while Grafana is used to visualise metrics through dashboards.

For LabLend, Prometheus and Grafana were selected as the monitoring technologies. The prometheus-flask-exporter package was integrated with the Flask application and, together with custom LabLend metrics, exposed application monitoring information through the /metrics endpoint. node-exporter was added to monitor the server's CPU, memory and disk.

A `prometheus.yml` configuration was created with three scrape jobs (`lablend`, `node` and `prometheus`, every 10 seconds), and Prometheus was added as another service in the Docker Compose configuration. Prometheus accesses the Flask service through `web:5000` on the Compose network and periodically scrapes the /metrics endpoint.

The Prometheus target page was used to verify that the LabLend application was UP. PromQL queries were used to analyse the application: `lablend_loans` shows requested, issued and overdue loans, and `rate(flask_http_request_total[1m])` shows HTTP requests by response status. Grafana opens on the provisioned dashboard "LabLend – Application & Server Monitoring" with app status, overdue loans, pending requests, units issued vs available, request rate, p95 latency, CPU, memory and disk.

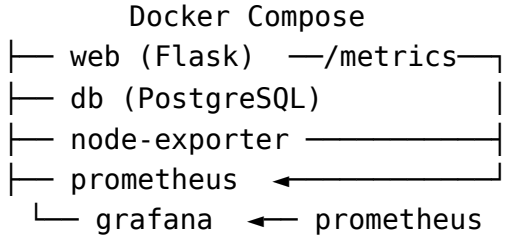


Figure 5.34: Monitoring services in the Docker Compose network

LO5: "Implement ML lifecycle management and automated workflows using MLflow, Airflow, and monitoring tools like Prometheus/Grafana/Nagios."

```
global:
  scrape_interval: 10s
  evaluation_interval: 10s

scrape_configs:
  # LabLend Flask application (HTTP + business metrics)
  - job_name: lablend
    metrics_path: /metrics
    static_configs:
      - targets: ["web:5000"]

  # Host CPU, memory and disk metrics
  - job_name: node
    static_configs:
      - targets: ["node-exporter:9100"]

  - job_name: prometheus
    static_configs:
      - targets: ["localhost:9090"]
```

Figure 5.35: Prometheus configuration

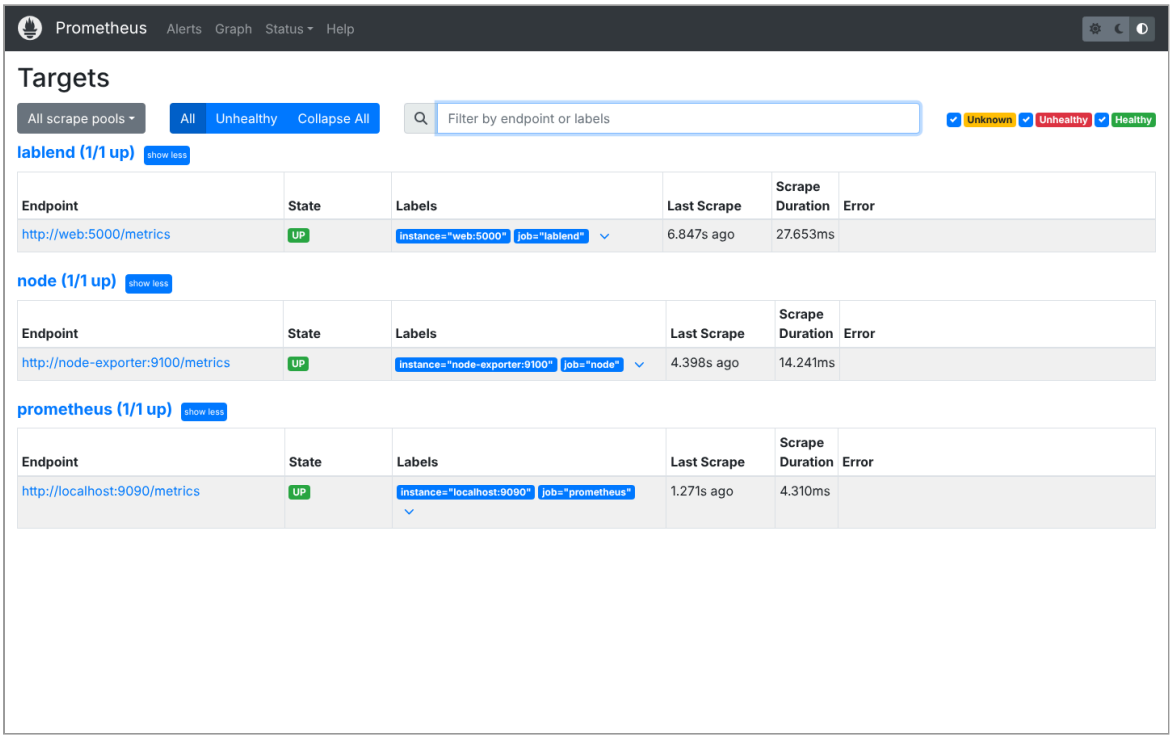


Figure 5.36: Prometheus target status showing lablend, node and prometheus UP

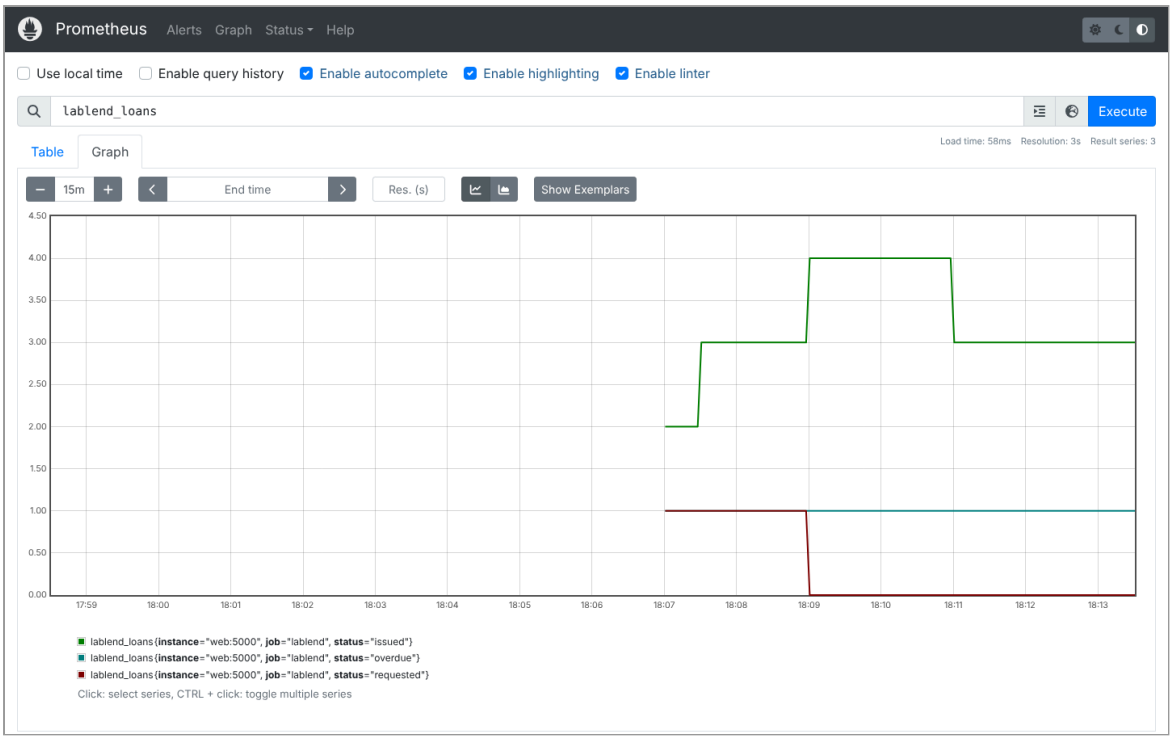


Figure 5.37: Prometheus query: lablend_loans by status

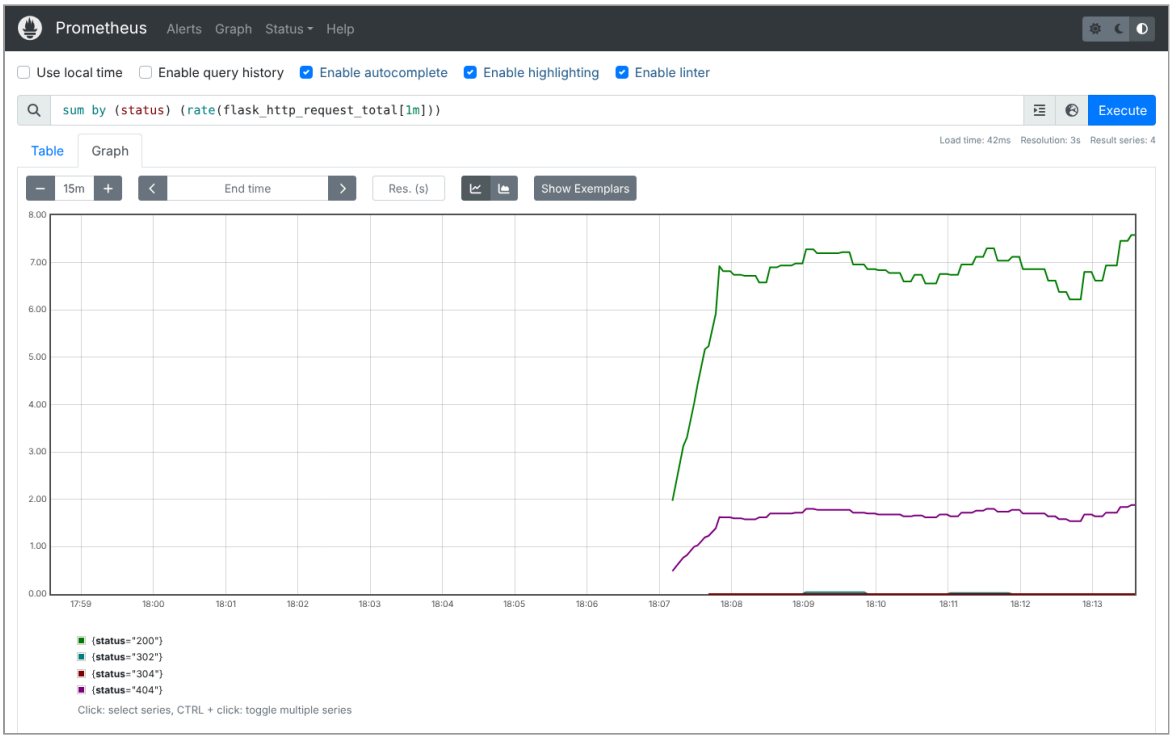


Figure 5.38: Prometheus HTTP request metrics: request rate by status

```
# HELP flask_exporter_info Information about the Prometheus Flask exporter
# TYPE flask_exporter_info gauge
flask_exporter_info{version="0.23.1"} 1.0
# HELP flask_http_request_duration_seconds Flask HTTP request duration in seconds
# TYPE flask_http_request_duration_seconds histogram
flask_http_request_duration_seconds_bucket{endpoint="api.api_equipment",le="0.005",method="GET",status="200"} 0.0
flask_http_request_duration_seconds_bucket{endpoint="api.api_equipment",le="0.01",method="GET",status="200"} 0.0
flask_http_request_duration_seconds_bucket{endpoint="api.api_equipment",le="0.025",method="GET",status="200"} 68.0
flask_http_request_duration_seconds_bucket{endpoint="api.api_equipment",le="0.05",method="GET",status="200"} 70.0
flask_http_request_duration_seconds_bucket{endpoint="api.api_equipment",le="0.075",method="GET",status="200"} 70.0
flask_http_request_duration_seconds_bucket{endpoint="api.api_equipment",le="0.1",method="GET",status="200"} 70.0
flask_http_request_duration_seconds_bucket{endpoint="api.api_equipment",le="0.25",method="GET",status="200"} 70.0
flask_http_request_duration_seconds_bucket{endpoint="api.api_equipment",le="0.5",method="GET",status="200"} 70.0
flask_http_request_duration_seconds_bucket{endpoint="api.api_equipment",le="0.75",method="GET",status="200"} 70.0
flask_http_request_duration_seconds_bucket{endpoint="api.api_equipment",le="1.0",method="GET",status="200"} 70.0
flask_http_request_duration_seconds_bucket{endpoint="api.api_equipment",le="2.5",method="GET",status="200"} 70.0
flask_http_request_duration_seconds_bucket{endpoint="api.api_equipment",le="5.0",method="GET",status="200"} 70.0
flask_http_request_duration_seconds_bucket{endpoint="api.api_equipment",le="7.5",method="GET",status="200"} 70.0
flask_http_request_duration_seconds_bucket{endpoint="api.api_equipment",le="10.0",method="GET",status="200"} 70.0
flask_http_request_duration_seconds_bucket{endpoint="api.api_equipment",le="+Inf",method="GET",status="200"} 70.0
flask_http_request_duration_seconds_count{endpoint="api.api_equipment",method="GET",status="200"} 70.0
flask_http_request_duration_seconds_sum{endpoint="api.api_equipment",method="GET",status="200"} 1.212138377998599
flask_http_request_duration_seconds_bucket{endpoint="auth.login",le="0.005",method="GET",status="200"} 74.0
flask_http_request_duration_seconds_bucket{endpoint="auth.login",le="0.01",method="GET",status="200"} 74.0
flask_http_request_duration_seconds_bucket{endpoint="auth.login",le="0.025",method="GET",status="200"} 75.0
flask_http_request_duration_seconds_bucket{endpoint="auth.login",le="0.05",method="GET",status="200"} 75.0
flask_http_request_duration_seconds_bucket{endpoint="auth.login",le="0.075",method="GET",status="200"} 75.0
flask_http_request_duration_seconds_bucket{endpoint="auth.login",le="0.1",method="GET",status="200"} 75.0
flask_http_request_duration_seconds_bucket{endpoint="auth.login",le="0.25",method="GET",status="200"} 75.0
flask_http_request_duration_seconds_bucket{endpoint="auth.login",le="0.5",method="GET",status="200"} 75.0
flask_http_request_duration_seconds_bucket{endpoint="auth.login",le="0.75",method="GET",status="200"} 75.0
flask_http_request_duration_seconds_bucket{endpoint="auth.login",le="1.0",method="GET",status="200"} 75.0
flask_http_request_duration_seconds_bucket{endpoint="auth.login",le="2.5",method="GET",status="200"} 75.0
flask_http_request_duration_seconds_bucket{endpoint="auth.login",le="5.0",method="GET",status="200"} 75.0
flask_http_request_duration_seconds_bucket{endpoint="auth.login",le="7.5",method="GET",status="200"} 75.0
flask_http_request_duration_seconds_bucket{endpoint="auth.login",le="10.0",method="GET",status="200"} 75.0
flask_http_request_duration_seconds_bucket{endpoint="auth.login",le="+Inf",method="GET",status="200"} 75.0
flask_http_request_duration_seconds_count{endpoint="auth.login",method="GET",status="200"} 75.0
flask_http_request_duration_seconds_sum{endpoint="auth.login",method="GET",status="200"} 0.038251582000061196
flask_http_request_duration_seconds_bucket{endpoint="None",le="0.005",method="GET",status="404"} 70.0
flask_http_request_duration_seconds_bucket{endpoint="None",le="0.01",method="GET",status="404"} 70.0
flask_http_request_duration_seconds_bucket{endpoint="None",le="0.025",method="GET",status="404"} 70.0
flask_http_request_duration_seconds_bucket{endpoint="None",le="0.05",method="GET",status="404"} 70.0
flask_http_request_duration_seconds_bucket{endpoint="None",le="0.075",method="GET",status="404"} 70.0
flask_http_request_duration_seconds_bucket{endpoint="None",le="0.1",method="GET",status="404"} 70.0
flask_http_request_duration_seconds_bucket{endpoint="None",le="0.25",method="GET",status="404"} 70.0
flask_http_request_duration_seconds_bucket{endpoint="None",le="0.5",method="GET",status="404"} 70.0
flask_http_request_duration_seconds_bucket{endpoint="None",le="0.75",method="GET",status="404"} 70.0
flask_http_request_duration_seconds_bucket{endpoint="None",le="1.0",method="GET",status="404"} 70.0
flask_http_request_duration_seconds_bucket{endpoint="None",le="2.5",method="GET",status="404"} 70.0
flask_http_request_duration_seconds_bucket{endpoint="None",le="5.0",method="GET",status="404"} 70.0
flask_http_request_duration_seconds_bucket{endpoint="None",le="7.5",method="GET",status="404"} 70.0
flask_http_request_duration_seconds_bucket{endpoint="None",le="10.0",method="GET",status="404"} 70.0
flask_http_request_duration_seconds_bucket{endpoint="None",le="+Inf",method="GET",status="404"} 70.0
flask_http_request_duration_seconds_count{endpoint="None",method="GET",status="404"} 70.0
flask_http_request_duration_seconds_sum{endpoint="None",method="GET",status="404"} 0.018581090997031424
flask_http_request_duration_seconds_bucket{endpoint="api.health",le="0.005",method="GET",status="200"} 73.0
flask_http_request_duration_seconds_bucket{endpoint="api.health",le="0.01",method="GET",status="200"} 73.0
flask_http_request_duration_seconds_bucket{endpoint="api.health",le="0.025",method="GET",status="200"} 73.0
```

Figure 5.39: Raw /metrics endpoint of the LabLend application



Figure 5.40: Grafana dashboard: application and server (CPU, memory, disk) monitoring

6. Testing

6.1 Automated Testing

Testing was incorporated into the Jenkins Continuous Integration pipeline. A pytest suite (`tests/test_app.py`) covers the health endpoint and API, registration and login, role checks, equipment requests, the loan-period and active-loan limits, approval, rejection and return, overdue detection, equipment management, CSV export and the metrics endpoint. The suite started with 19 tests and grew to 22 with the CSV export feature. The Jenkinsfile runs the tests inside the `test` stage of the Dockerfile and publishes a JUnit report, so the tests are executed as part of the CI workflow rather than only as a manual local activity.

6.2 CI Verification

The CI workflow was verified through the sequence Developer → Git Push → GitHub → SCM polling → Jenkins → Build Test Image + Unit Tests → SUCCESS / FAILURE. Build #3 confirmed that a failing test stops the pipeline: the Docker build, deployment and smoke test stages were skipped. The deployment extension in Experiment 5 additionally required the testing stage to succeed before Docker image building, Docker Compose deployment and the /health smoke test proceeded.

6.3 Monitoring Verification

Monitoring verification was performed by checking the Prometheus target page and confirming that the LabLend application, node-exporter and Prometheus were UP. PromQL was then used to inspect loan and HTTP request metrics, and the Grafana dashboard was checked for application and server panels. This verified that monitoring was integrated with the deployed application.

6.4 Testing Evidence

This report contains screenshots and command output of Git, Docker, Docker Compose, Jenkins and Prometheus/Grafana activities. These provide practical evidence of the implementation and verification stages described in the experiments. Full Jenkins console logs of builds #1–#4 are kept in `docs/evidence/` in the repository.

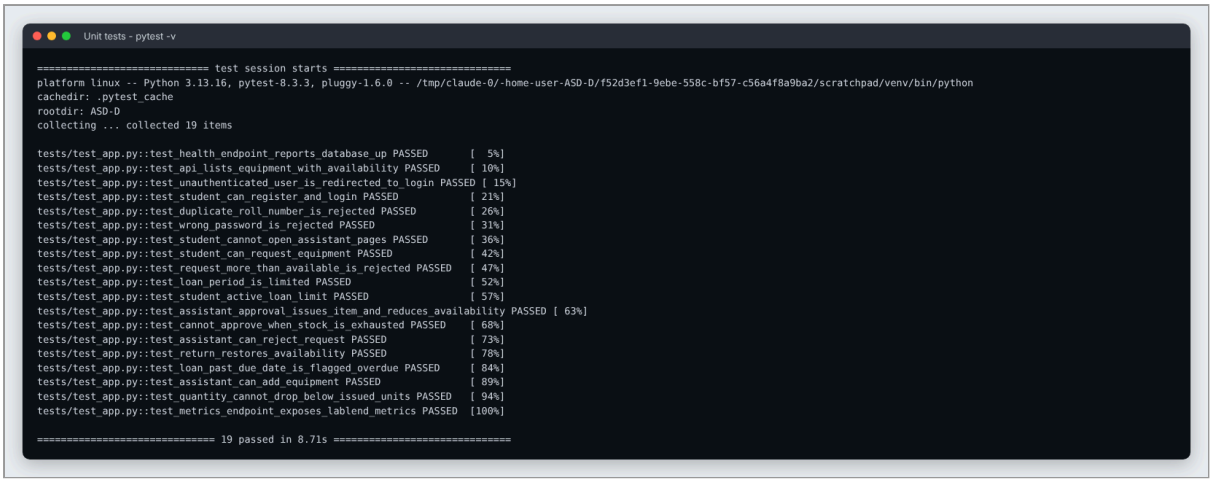


Figure 6.1: Automated testing: pytest unit tests passing



Figure 6.2: CI evidence: Jenkinsfile stage commands (build, test, image build, deploy, smoke test) run from the Jenkins image

7. Implementation:

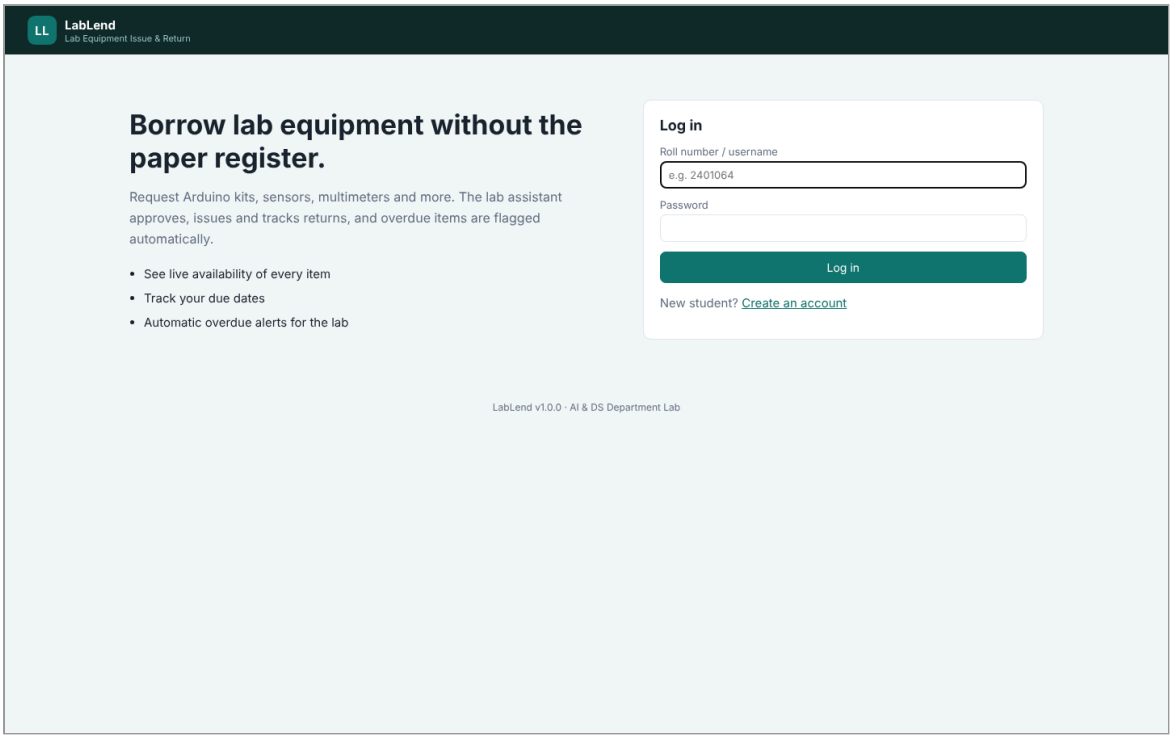


Figure 7.1: LabLend login page

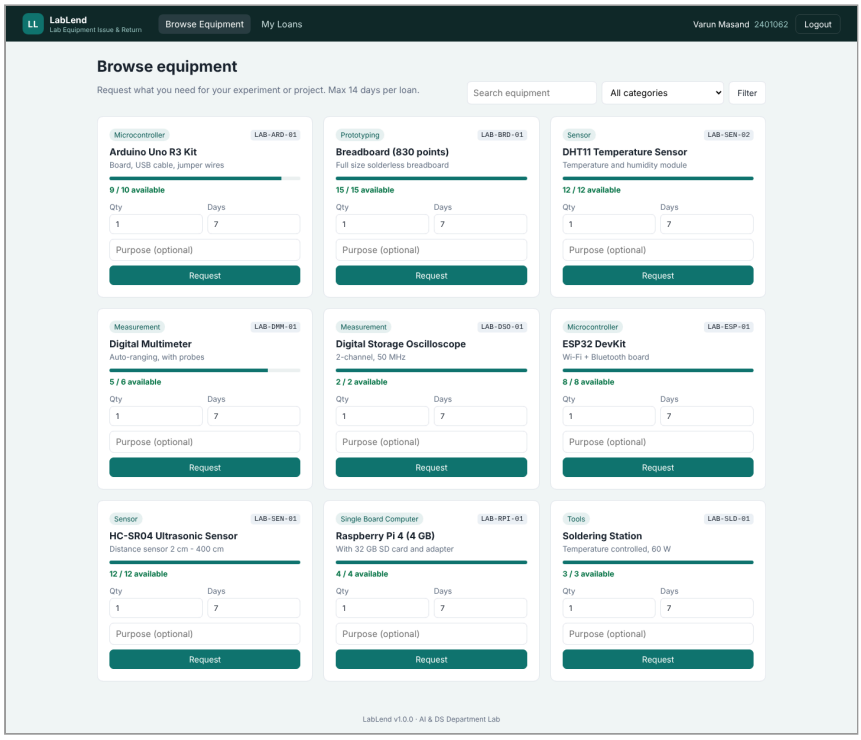


Figure 7.2: Student: browse equipment with live availability

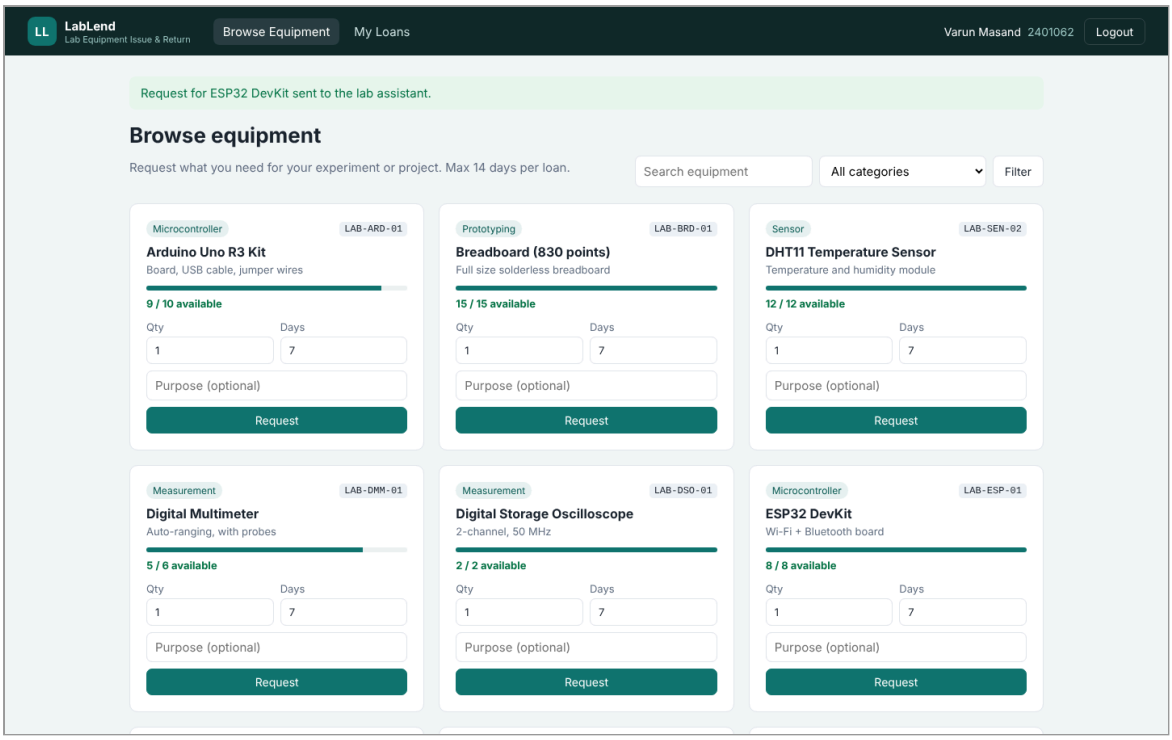


Figure 7.3: Student: equipment request sent to the lab assistant

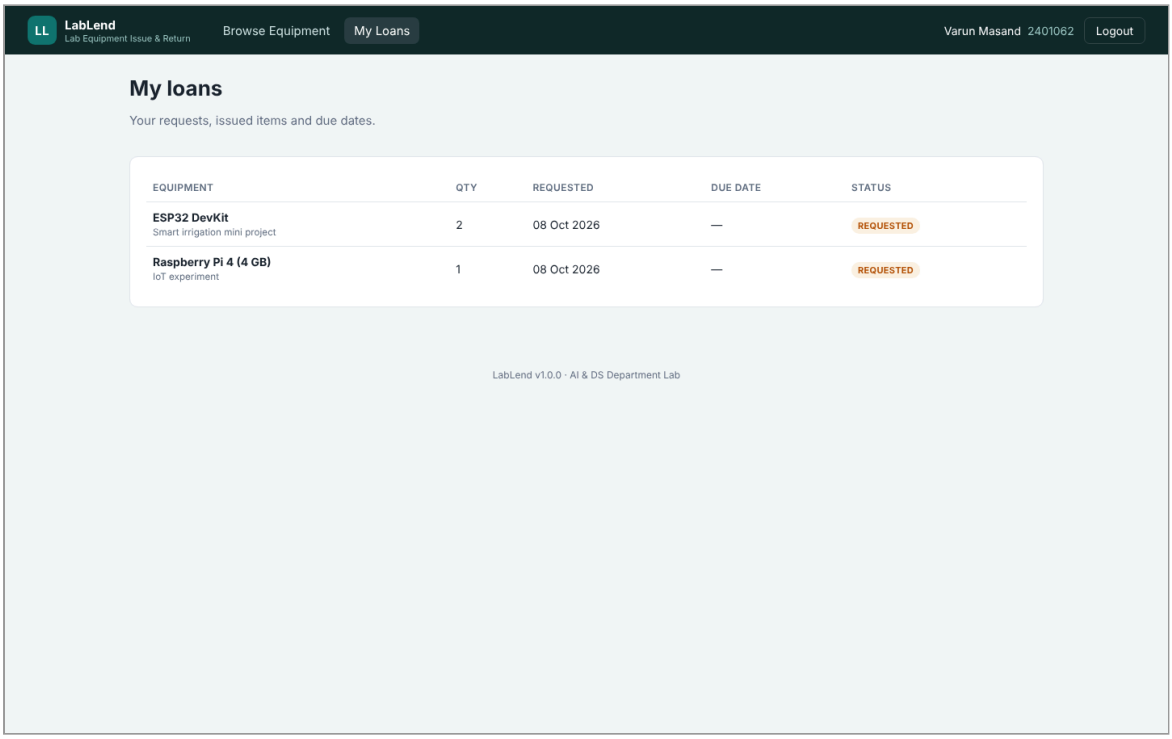


Figure 7.4: Student: My Loans with statuses and due dates

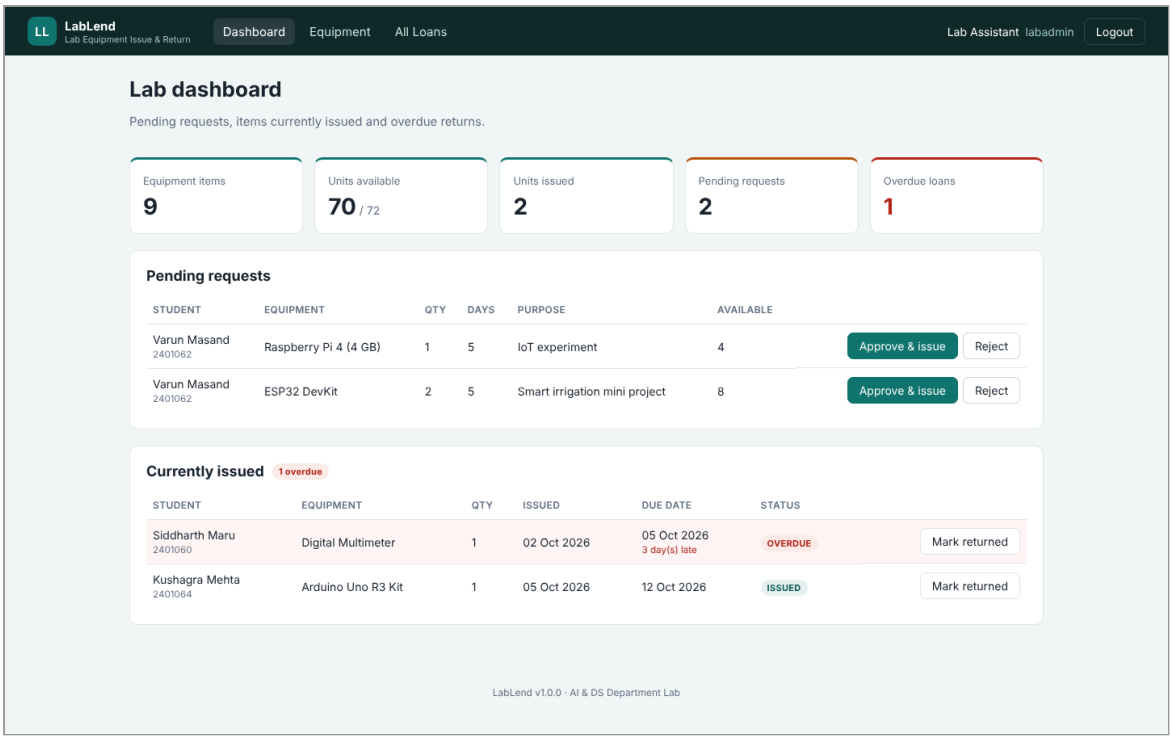


Figure 7.5: Lab assistant dashboard: pending, issued and overdue

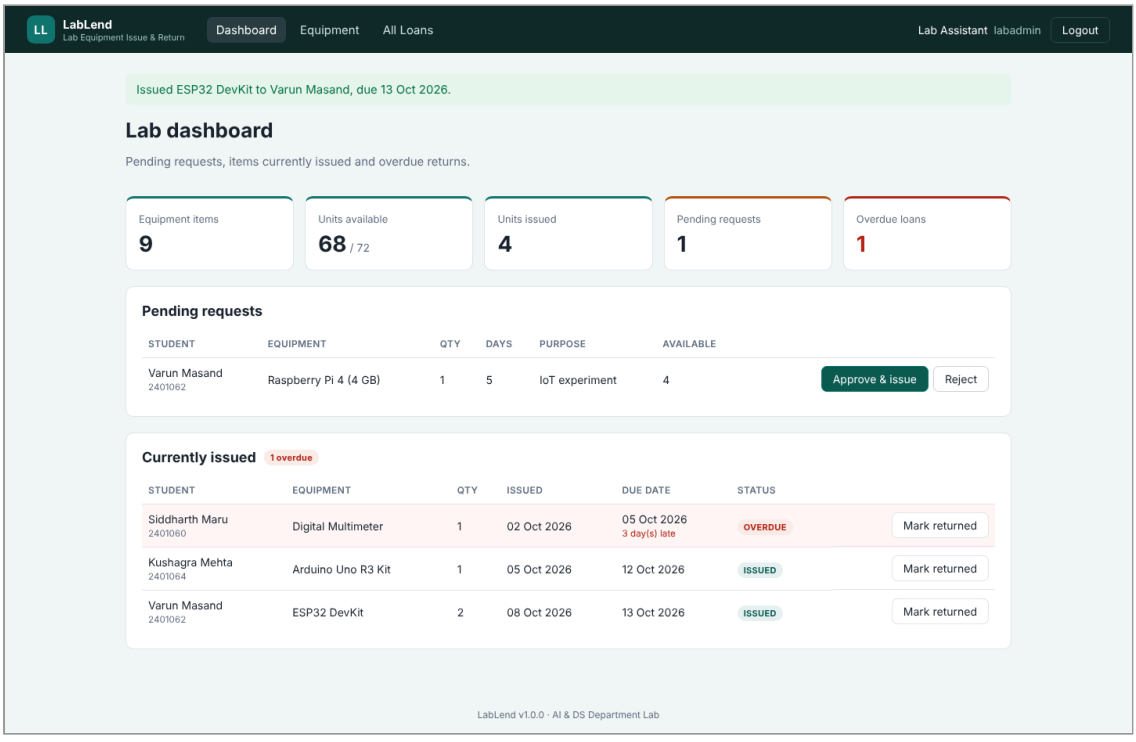


Figure 7.6: Lab assistant: request approved and issued with due date

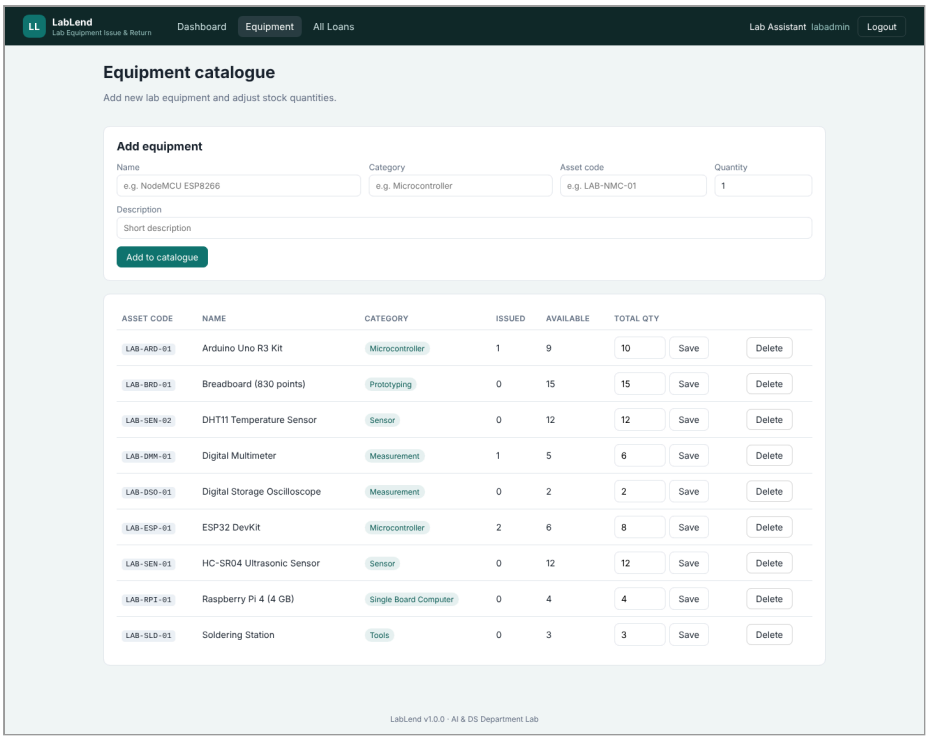


Figure 7.7: Lab assistant: equipment management

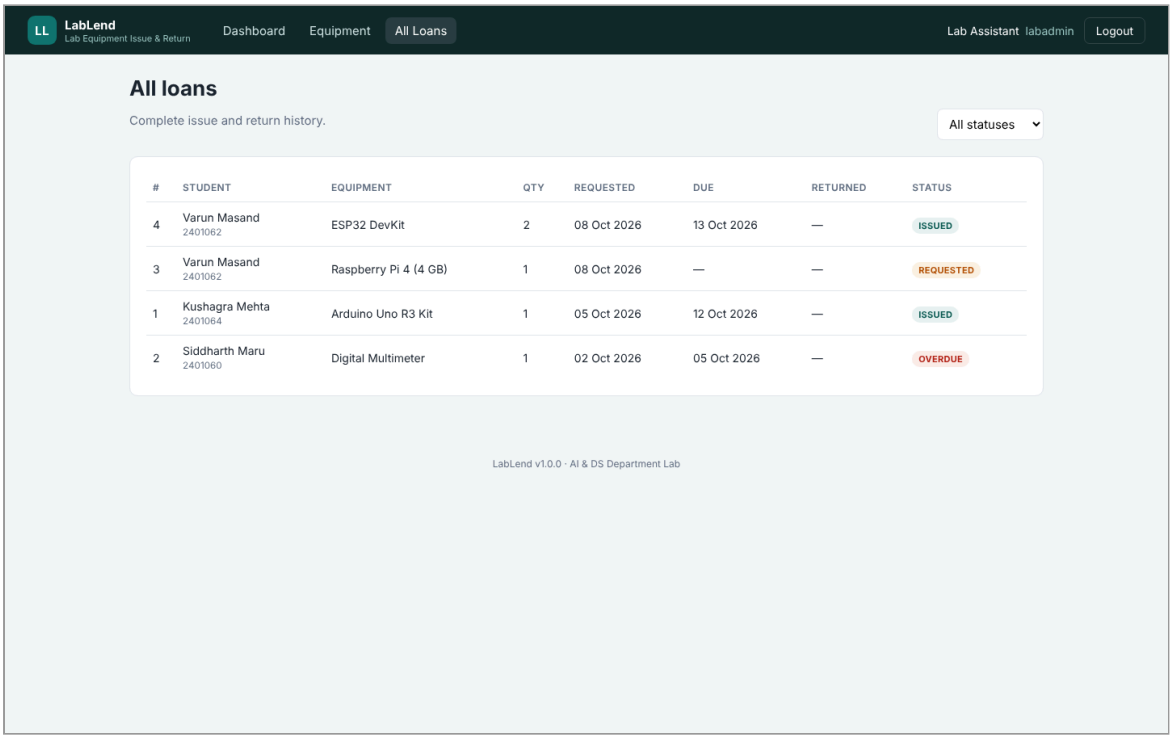


Figure 7.8: Lab assistant: full loan history

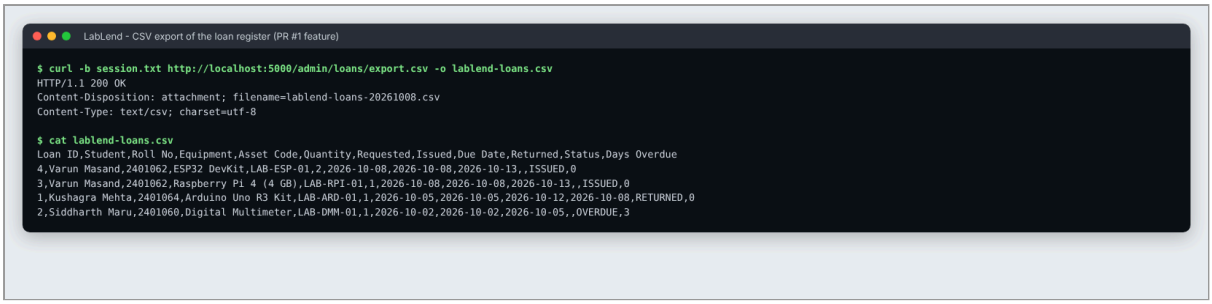


Figure 7.9: Downloaded CSV export of the loan register

8. Results and Discussion

8.1 Results

Area	Result
Version control	LabLend source code was maintained in Git and pushed to GitHub; feature branches, a pull request and merges were used for feature development, and a faulty change was rolled back with git revert.
Containerization	The Flask application was built into a multi-stage Docker image and executed as a container with port 5000 mapped to the host; the container reported healthy.
Multi-service deployment	Docker Compose ran the Flask application, PostgreSQL, Prometheus, node-exporter and Grafana as interconnected services with persistent storage and health checking.
Continuous Integration	Jenkins was connected to GitHub through SCM polling and executed the build and pytest stages automatically after changes (22 tests passing).
Automated deployment	The Jenkins pipeline was extended to build the Docker image, deploy the application using Docker Compose and run a /health smoke test after successful testing; a failing build blocked deployment.
Agile workflow	The Jira Scrum project (SCRUM) holds 5 epics and 19 stories/tasks (54 points) in three sprints, with two dependencies and a To Do, In Progress, In Review, Done workflow.
Continuous monitoring	Prometheus collected application metrics through /metrics and server metrics through node-exporter, supported PromQL-based analysis, and Grafana visualised them.

Table 8.1: Summary of project results

8.2 Discussion

LO6: “Develop and demonstrate an end-to-end DevOps pipeline mini-project integrating all stages of DevOps from code commit to deployment and monitoring.”

The project demonstrates that the individual laboratory experiments can be combined into one software-development lifecycle. Git and GitHub provide the source-control foundation; Docker and Docker Compose provide a consistent and multi-service execution environment; Jenkins automates verification and delivery; Jira connects development work with Agile planning; and Prometheus with Grafana extends the lifecycle into continuous monitoring.

The integrated workflow reduces repetitive manual operations and provides a clear relationship between source-code changes, testing, deployment, Agile task tracking and monitoring. The failed build #3 and the reverted build #4 show the value of this connection in practice: a change that broke an agreed business rule was caught by the tests before deployment, and the rollback was deployed automatically. The project transformed the individual laboratory concepts into a connected practical DevOps workflow.

9. Conclusion

The development of LabLend – Lab Equipment Issue and Return Tracker provided practical experience in applying software-development and DevOps concepts to a single integrated application.

Git and GitHub provided structured version control, commit history, branching, pull requests and rollback. Docker packaged the application and its dependencies into a portable containerized environment, while Docker Compose coordinated Flask, PostgreSQL and the monitoring services as interconnected services with networking and persistent storage.

Jenkins demonstrated how Continuous Integration can automatically verify changes through testing and how a CI/CD pipeline can further automate Docker image building, application deployment and a post-deployment smoke test. Connecting GitHub and Jenkins showed how a source-code change can automatically initiate the delivery pipeline, reducing repetitive manual operations and improving consistency.

Jira introduced the practical application of Agile and Scrum principles by representing software features as epics, stories and tasks, planning them into sprints and moving them through stages of development, review and completion.

Prometheus and Grafana demonstrated that the DevOps lifecycle does not end after deployment. By exposing Flask and business metrics and collecting server metrics, application traffic, HTTP status codes, request latency, loan counts and CPU, memory and disk usage can be observed to understand the behaviour and health of a running system.

Overall, the LabLend mini project helped demonstrate how version control, application development, containerization, multi-service deployment, CI/CD automation, Agile project management and continuous monitoring work together as parts of a complete software-development lifecycle.